

C25 — AI Product Management: Building, Evaluating and Governing

1. What interviewers test

"How would you evaluate the quality and safety of on-device summarization?" "Your reply suggestions are accepted 45% of the time. Is the feature working?" "Can we trust a language model to grade another model's answers?" "How much autonomy should a refund agent get, and how do you decide when to give it more?" "What is the moat of an AI writing product if the underlying models keep improving?" These questions go beyond understanding how AI behaves (C24). They ask whether you can build an AI product responsibly: define quality before building, measure it honestly, run experiments that account for the quirks of AI, design safety in layers, earn autonomy with evidence, make the economics work, handle regulation, and find a durable advantage.

Where it shows up. Inferred from the round structures candidates commonly report: AI product-sense, analytical and strategy questions at Google, Meta, Microsoft, Amazon, Apple, Netflix, Stripe and others, increasingly with an explicit evaluation, safety or economics component [unverified: candidate-reported]. On the job it is the work of PMs who own an AI feature: writing evaluation plans, reading results, deciding launches, setting guardrails and pricing.

What is scored. Whether you treat evaluation as the center of AI product work Inferred. Interviewers listen for an evaluation plan built before the feature (a test set from real inputs, a rubric, human review and the cautious use of model graders); metrics that separate model quality from product value; experiments that handle variability, novelty and over-reliance; safety designed in layers with adversarial testing; autonomy that is earned with evidence; cost per task and margins; a sensible view of regulation by risk; and an honest answer on defensibility.

Six observable behaviors:

1. You design the evaluation before the feature: a test set, a rubric, a target and a way to run it on every change.
2. You size evaluation sets and read their uncertainty, and you check that raters agree beyond chance.
3. You separate model quality from product value, and watch for metrics that reward over-reliance.
4. You design safety as layers, test it adversarially, and plan the response when it fails.

5. You promote autonomy only on evidence, with demotion rules.
6. You reason about cost per task, margins, regulation by risk tier, and what makes the product defensible.

The failure this prevents. Six patterns end AI-building answers: "we'll test it after we build it," an evaluation of ten demos, an acceptance rate taken as quality, a model judge trusted blindly, safety as one filter, and autonomy granted because the demo went well.

2. The concept from first principles

C25.1 Eval-first development: offline sets, human review, model-graded evals

An **eval** is a repeatable test set and scoring method used to measure the quality of an AI system's output. In AI products the evaluation is the specification: it says what "good" means in concrete cases (FW-C25-01). Build it before the feature, because it decides what you build, lets you compare approaches fairly, and catches regressions when anything changes.

The parts.

1. *Cases*. Sample real inputs from the intended use (or the closest available), not only clean examples. Include the long tail: ambiguous requests, typos, other languages, adversarial inputs, questions with no answer, and the cases where the feature should decline. Cover each slice that matters (user segment, language, topic, input length) with enough cases to read separately.
2. *A rubric*. Written criteria for a good output, specific enough that two reviewers would mostly agree: for a summary, "every claim is supported by the source," "no urgent detail is dropped," "tone fits." Use a few criteria with clear scales and examples of each score.
3. *Judges*. People first (domain experts for specialized tasks); rules where checkable (does the answer cite a source from the retrieved set; does the code run); and, with care, a model.
4. *A target and a gate*. A pass bar tied to the cost of error (C24.1): for example, fewer than 2% of summaries with an unsupported claim, and no failures in the worst category. The eval runs on every change to the model, prompt, retrieval or data, and a launch needs a pass.

How many cases? The uncertainty of a measured rate shrinks with the number of cases (C12). To estimate an error rate of about p within $\pm\delta$ at 95% confidence you need about $n = 1.96^2 \times p \times (1 - p) \div \delta^2$ cases. For $p = 3\%$ and $\delta = 1.5$ points that is about 497, so roughly 500; for $p = 2\%$ and $\delta = 1$ point, about 753; for $p = 5\%$ and $\delta = 2$ points, about 456 (FW-C25-02). Each slice you want to read separately needs its own sample, so ten languages need ten times as many. Small sets (30 cases) can find problems and cannot certify a rate: with 30 cases and no errors the 95% upper bound on the true error rate is still about 10% (the "rule of three": about $3 \div n$ [V: arithmetic checked]).

Human review and agreement. Inter-rater agreement is the extent to which independent reviewers give the same judgment, corrected for chance. Raw agreement misleads when most cases are good: two reviewers who each flag 9% of items as bad and agree on 94 of 100 items have 94% raw agreement, but the agreement expected by chance is $0.09 \times 0.09 + 0.91 \times 0.91 = 83.6\%$; the chance-corrected measure (Cohen's kappa) is $(0.94 - 0.836) \div (1 - 0.836) = 0.63$ [V: arithmetic checked]. Moderate agreement means the rubric is ambiguous: tighten it with examples, then re-measure (Worked example 1). Where reviewers disagree, discuss and record the reasoning; the disagreements show where the quality definition is unclear.

Model-graded evals. A **model-graded eval** uses a language model to score another model's outputs against a rubric. It is fast and cheap, and it has known weaknesses: it may prefer longer or more fluent answers, favor outputs resembling its own style, miss subtle factual errors, and be wrong in ways that correlate with the system under test. Use it only after calibrating it against human judgments on a sample: measure its precision and recall on the failures humans found (C23.3). A judge that agrees with humans 90% of the time can still miss 40% of the real failures if failures are rare (Worked example 2). A common pattern: the judge screens everything, humans review what it flags and a random sample of what it passes, and the sample estimates what the judge misses.

Regression tests. A **regression test** is a test that checks that a change has not broken behavior that used to work. For AI products the suite is the eval, run on every change and read by slice, since an average can hold steady while one category collapses (CS-C25-01). Keep a "never again" set of every failure found in production.

Regional Lens: India. Evaluation must cover the languages and scripts users actually use, including mixed-language text and transliteration. Reviewers who read the language are scarce for some languages, quality often differs sharply between languages, and a feature that passes in English can fail elsewhere Inferred [unverified: author's synthesis; no source checked]. The consequence for a PM: build separate test sets and rubrics per language, recruit reviewers with local fluency and cultural context, report results per language and not as one average, and set per-language launch gates.

Counterexample. A team builds an assistant, tries it on ten examples that look good, ships it and then builds an evaluation after complaints. The post-launch eval shows 14% of answers contain an unsupported claim; the team has no baseline for what changed, and every fix is a guess. A team that had assembled 500 real questions and a rubric first would have seen the 14% before launch, set a target, and known whether each change helped.

Why it works. The eval turns opinions about quality into numbers, and makes change safe. **When not to over-invest.** For a low-stakes, reversible feature a small eval (50 to 100 real cases) and monitoring may be enough; scale the eval to the cost of error.

C25.2 A worked eval design

Section 5 gives a full eval design (Worked example 1) and a model-judge calibration (Worked example 2). The steps (FW-C25-01):

1. **Define the task and its failure types** with examples, including what the feature must decline.
2. **Collect cases** from real or realistic inputs, stratified by slice; add adversarial and unanswerable cases; remove duplicates; keep a held-out portion that engineers do not tune against.
3. **Write the rubric** with two or three criteria and scored examples; pilot it on 30 cases.
4. **Label** with two independent reviewers on a sample; compute agreement; revise the rubric until agreement is acceptable; resolve disagreements.
5. **Baseline:** score the current approach (and a simple alternative) to learn where you start.
6. **Set the pass bar** from the cost of error, with per-slice minimums and zero tolerance for the worst failures.
7. **Automate:** run on every change; report overall and per slice with intervals.
8. **Monitor in production:** sample live traffic for review, track user signals, and add every found failure to the "never again" set.

Beware overfitting to the eval. If engineers tune prompts and models against the same cases repeatedly, scores rise without real improvement (C23.2). Keep a hidden test set, refresh it from live traffic, and watch for a gap between eval and production performance.

C25.3 AI metrics and online experimentation

C23.7 gave a metric stack for ranked systems; for AI features use the same layers (FW-C25-03):

Layer	Examples	Danger if alone
Output quality (eval)	Faithfulness, helpfulness, error rate by category, appropriate refusals	Measures the model, not the product
Product behavior	Task success, edits to the output, acceptance, escalation to a person, time to complete	Acceptance can reward over-reliance
Business	Retention, conversion, deflected support contacts, revenue per user	Slow; confounded
Cost and speed	Cost per task, 95th-percentile latency	Cheap and fast can be wrong
Safety and trust	Reports, harmful-output rate, complaints, opt-outs	Rare events need large samples

The acceptance trap. A high acceptance rate for suggestions can mean the feature is good or that users trust it too much. Measure *accepted and wrong*: sample accepted outputs for errors, and track later corrections and complaints. Worked example 3 shows that at an 8% error rate among accepted suggestions the feature costs more time than it saves, and that the break-even error rate is about 3.7%.

Experiments on AI features (C14, C17). Outputs vary run to run, so measure at the level of the user or task over many cases, not single examples. Reading effects by week matters because novelty draws early use and users adapt (they learn to trust, to check, or to stop using it) (C14.4). Use a holdout for long-term effects. Offline evals gate; online tests judge. Run **shadow mode**: a deployment in which the new system runs on live inputs and its outputs are logged and judged but not shown to users, so you can measure quality on real traffic without risk. Then a canary to a small group (C17.2).

Guardrail metrics. Safety, cost and latency must not worsen beyond margins, as in C14.5, with extra care for rare harms that need large samples: to see a harm occurring at 1 in 10,000 you need on the order of tens of thousands of cases per arm.

C25.4 Safety, guardrails, abuse; designing for failure

Safety for AI products is layered defense (the Swiss-cheese idea, C17, mental models, section 9). A **guardrail** is a control placed around a model to keep its behavior within limits: filters on inputs and outputs, rules about what it may do, limits on tools and permissions, and escalation to people. FW-C25-04 lays out the layers.

1. **Scope and design.** Limit what the feature does (answer from documents, no medical advice), and design out the dangerous capabilities (no money movement without approval).
2. **Input defenses.** Detect and handle prompt injection, abusive and out-of-scope requests; limit the rate per user (**rate limiting**, C22.4).
3. **Model behavior.** Instructions, training and refusal behavior that decline harmful requests, tested for over-refusal as well as under-refusal (a safety filter is a classifier with precision and recall, C23.3).
4. **Output defenses.** Filters for harmful or private content, grounding checks (is each claim supported?), format constraints.
5. **Permissions and actions.** Least privilege, approvals, reversibility, logging (C24.4).
6. **Human review and escalation.** Queues for flagged outputs, a path for users to report, and authority to restrict behaviors quickly.
7. **Monitoring and response.** Sample live traffic, track harm rates and drift, run incident response with a kill switch and a postmortem (C17.5).

Red-teaming. **Red-teaming** is deliberate adversarial testing in which people try to make a system fail or cause harm, to find weaknesses before real users do. Use internal and external testers, a harm taxonomy (dangerous advice, harassment, sexual content, privacy, deception, manipulation, security), and track the attack success rate per category. New attack methods appear continually, so red-teaming is continuous work.

Layer arithmetic. If an input filter stops 80% of attack attempts, the model refuses 90% of those that pass, and an output filter catches 70% of what remains, then the share of attempts that succeed is $0.2 \times 0.1 \times 0.3 = 0.6\%$. At 100,000 attempts a day that is 600 successful attacks; improving the output filter to 90% cuts that to 200, and limiting repeat attackers to halve the attempts cuts it to 100 (Worked example 4). Each layer is imperfect, and their product is much better than any one.

Drift. **Drift** is the change over time in inputs, user behavior or the world that makes a deployed model perform worse than when it was evaluated. Detect it by monitoring input distributions, the share of outputs flagged, the eval score on fresh live samples and user signals; refresh the eval and retrain or re-prompt as needed.

Designing for failure. Assume the model will sometimes fail and plan the user's path: a fallback that shows the original content, a handoff to a person, a clear message, and a way to report. Practice incident response before launch: who can turn the feature off, how fast, and who tells users.

C25.5 Autonomy and trust progression for agentic products

C24.4 introduced the autonomy ladder (suggest, draft, act with approval, act autonomously). Here is how to move along it with evidence (FW-C25-05).

Earned autonomy. Start each capability at the lowest level that is useful. Promote it only when (1) the error rate at the current level is measured on enough cases and below a bar set from the cost of error; (2) the action is reversible or capped in amount; (3) monitoring and a kill switch exist; and (4) users' overrides are low. Demote on a rule written in advance: if errors exceed a threshold in a week, or one severe incident occurs, drop a level and investigate.

How much evidence? To claim that an error rate is below 1% with 95% confidence after observing no errors, you need about $3 \div 0.01 = 300$ supervised cases (the rule of three [V: arithmetic checked]); if you want to claim below 0.2%, about 1,500. If errors occur, you need more. Evidence should be gathered per category of action and per amount band, since reliability on small refunds says little about large ones.

Limits that make autonomy safe. Caps on value (refunds under \$20), counts (ten actions an hour), scope (only the customer in the conversation), time (revoke after a session) and audit trails (every action logged with the reason and the evidence). Autonomy without limits is a bet on the model's behavior; limits turn it into a bounded risk.

Trust is two-sided. Users' trust should track reliability: tell people what the agent has done, make actions easy to review and undo, and avoid presenting confident prose over uncertain actions.

C25.6 Economics: cost per task, pricing, margins

AI features have a real, variable cost per use (C24.7) that changes how products are priced and how margins behave (FW-C25-06; C18.6 gave the general method).

Cost per task. Cost per task = tokens in × input price + tokens out × output price, plus retrieval, tools, human review and support. Track it per task type and per user, not on average.

Heavy users. Usage is uneven. Suppose a \$20 per-seat monthly add-on: an average user runs 300 tasks at \$0.03 each, \$9, plus \$2 of other cost: a \$9 margin (45%). A heavy user running 1,500 tasks costs \$45 plus \$2: a loss of \$27. If 10% of users are heavy, the blended cost is $0.9 \times \$11 + 0.1 \times \$47 = \$14.60$ and the margin falls to \$5.40 (27%) (Worked example 5). Flat pricing with unlimited use exposes the product to its heaviest users.

Levers. Route easy tasks to cheaper models (if 60% of tasks cost a tenth as much, the average task cost falls from \$0.03 to \$0.0138 and the blended margin rises to about \$12.20, 61%); cache; shorten prompts; limit outputs; set usage tiers or included allowances with metered overage; price by outcome where value is measurable (a resolved ticket); and design features that reduce unnecessary calls.

Pricing models. Per seat (simple, predictable, exposed to heavy users), per usage (aligns cost and price, unpredictable for customers), per outcome (aligns with value, needs measurable outcomes and trust in measurement), and bundling into an existing product (hides the cost and must be covered by the product's margin)

[V: All four patterns exist in vendor pricing (2025 to 2026 reports): per seat (Microsoft 365 Copilot), per usage or conversation (Salesforce Agentforce, credit-based plans), per outcome (Intercom's Fin at about 0.99 dollars per outcome, and Zendesk per automated resolution; Intercom renamed itself Fin in 2026, agreed on 15 June 2026 to be acquired by Salesforce, and Salesforce announced the completion on 10 September 2026, per Salesforce's releases), bundling into suites (Microsoft); prices change often and are not quoted in the book].

Do not assume costs fall on their own. Prices for the same capability have tended to decline over time [V: Stanford HAI, AI Index 2025 (with Epoch AI data): GPT-3.5-level performance fell from 20 to 0.07 dollars per million tokens, November 2022 to October 2024; falls of 9 to 900 times per year by task; a fixed-capability price, not frontier-model pricing], and usage and expectations rise with them; plan margins on current costs and treat decline as upside.

C25.7 Regulation and compliance overview by region

[as of 2026-10] This section is an overview for orientation, not legal advice; involve counsel and check current texts, because rules and dates change quickly. The practical approach (FW-C25-07) is to classify each AI use by risk and apply the controls that tier requires.

Risk tiers. A **risk tier** is a category of AI use ranked by the potential for harm, which sets the obligations on its provider and deployer (as in the risk-based approach of the European Union's AI Act). A typical ordering: *prohibited* uses; *high-risk* uses (for example employment decisions, credit, education, essential services, safety components); *limited-risk* uses that require transparency (chatbots must disclose that the user is talking to a machine, AI-generated content may need labeling); and *minimal-risk* uses (a spam filter) with no special obligations.

Regions (as of 2026-10, check each).

- *European Union.* The AI Act entered into force in August 2024 and applies in phases: prohibited practices since February 2025; obligations for providers of general-purpose AI models since August 2025; transparency obligations for chatbots and generated content (Article 50) from 2 August 2026, with a transition to 2 December 2026 for machine-readable marking by generative systems already on the market; and the high-risk obligations, originally due 2 August 2026, deferred by Regulation (EU) 2026/1744 (the Digital Omnibus on AI, in force since 27 July 2026) to 2 December 2027 for stand-alone high-risk systems and 2 August 2028 for AI embedded in regulated products, with two new prohibitions (generating non-consensual intimate imagery and child sexual abuse material) from 2 December 2026 [V: Regulation (EU) 2026/1744 on EUR-Lex (title, recital 38); legal summaries for the dates]. Data protection law (the GDPR) also applies, including rules on solely automated decisions.
- *United States.* There is no single comprehensive federal AI statute as of mid-2026 [V: law-firm and industry summaries through July 2026; not rechecked for October 2026]; Executive Order 14179 of January 2025 was followed by an executive order of 11 December 2025, "Ensuring a National Policy Framework for Artificial Intelligence," which directs the Department of Justice to challenge state AI laws and does not itself preempt them [V: law-firm and industry summaries, 2026]; federal agencies apply existing laws (consumer protection against deceptive claims, anti-discrimination in employment and credit), and several states have passed or proposed AI laws, with dates that have shifted: Texas's Responsible AI Governance Act and California's frontier-model transparency law (SB 53) took effect on 1 January 2026, and Colorado, whose original AI Act had been postponed to 30 June 2026, repealed and replaced it on 14 May 2026 with a narrower law on automated decision-making technology that takes effect on 1 January 2027 [V: law-firm alerts, May to July 2026]. Some cities regulate specific uses, such as New York City's Local Law 144, enforced from 5 July 2023, which requires annual independent bias audits of automated employment tools [V: NYC Local Law 144: annual independent bias audit, published summary, notice; enforced from 5 July 2023].
- *India.* The Digital Personal Data Protection Act of 2023, with rules notified on 13 November 2025, with the Data Protection Board provisions effective at once, consent-manager provisions from 13 November 2026 and the main obligations from 13 May 2027, governs personal data [V: law-firm summaries of the DPDP Rules]; India has no standalone AI statute, the Ministry of Electronics and Information Technology issued non-binding AI Governance Guidelines on 5 November 2025, and amendments to the IT Rules effective 20 February 2026 require platforms to label and trace synthetically generated content and shorten takedown times [V: law-firm summaries; Hogan Lovells, 2026].
- *China.* Rules apply to recommendation algorithms, synthetic media and generative AI services, with requirements on content and registration [V: China: algorithm recommendation provisions (effective 1 March 2022), deep synthesis provisions (10 January 2023), generative AI interim measures (15 August 2023); filing and security assessment duties]. Two further instruments followed: labeling measures for AI-generated content, effective 1 September 2025 with a mandatory national standard, and interim measures for anthropomorphic (companion-style) AI interaction services, issued

on 10 April 2026 and effective 15 July 2026 [V: law-firm and legal-directory summaries, 2025 to 2026] .

- *Elsewhere.* The United Kingdom follows a non-statutory, principles-based approach through existing regulators, with no standalone AI statute as of mid-2026, and others take similar routes [V: UK: March 2023 white paper, five non-statutory principles applied by existing regulators; no AI statute as of mid-2026 (RPC, 1 June 2026)]; the May 2026 King's Speech announced a Regulating for Growth Bill with cross-economy sandbox powers and not a dedicated AI Act, and a law-firm note of 14 September 2026 says a dedicated AI bill had still not appeared [V: RPC; Lewis Silkin, 14 September 2026] .

What a PM does. Classify the use by tier and region; keep documentation (what the system is for, its data, its evaluation results, known limits, and monitoring), often called a **model card** when it describes a model: a short document stating a model's intended use, data, evaluation results, limitations and risks; build in required transparency (disclose AI interaction, label generated content, offer human review for significant decisions); keep records for audit; and bring in legal and policy early. A feature that is high-risk in one market may need a different design or a delayed launch there.

C25.8 Defensibility

If models are available to everyone, what makes an AI product hard to copy? Use the sources-of-advantage test (FW-C18-05) with an AI lens (Inferred):

- *Proprietary data and feedback loops:* data that others cannot get and that improves the product with use (but beware that general models may learn the task without your data).
- *Distribution:* access to users through an existing product or channel.
- *Workflow integration and switching costs:* the product sits inside how people work, with their data, templates and habits.
- *Evaluation capability:* a mature test set and monitoring let you improve faster and ship safely; competitors starting from nothing are slower.
- *Trust and brand:* in domains where errors are costly, a track record of reliability and accountability.
- *Cost position:* routing, caching and fine-tuning that deliver the same quality at lower cost.
- *Weak sources:* a clever prompt (easy to copy), the choice of model (others can switch), and a thin wrapper around a general model.

A defensible answer says what compounds (data, workflow, trust) and what does not, and what you would build toward as models improve.

C25.9 Answer architecture for AI product questions

Section 4 gives the full architecture. In outline: define the user problem and the cost of error; design the evaluation before the feature; choose metrics across layers and watch for acceptance traps; design safety in layers and test adversarially; set autonomy with evidence; compute cost per task and margins; note the risk tier and regulation; and say what is defensible.

C25.10 Worked examples

Section 5 gives five worked examples: an eval design with sample size and rater agreement, a model-judge calibration, an online experiment with the acceptance trap, layered safety arithmetic, and unit economics for an AI add-on. Section 11 adds ten questions with model answers and follow-ups.

3. Frameworks

Eight frameworks, set out below. The AI fit test (FW-C24-01), the cost and latency sheet (FW-C24-02), the AI failure-mode map (FW-C24-03), the autonomy ladder (FW-C24-05), the AI UX checklist (FW-C24-06), the experiment plan for ranking (FW-C23-07), the sources-of-advantage test (FW-C18-05) and the unit-economics sheet (FW-C18-06) are reused, not repeated.

FW-C25-01 · The eval design card · Original

- **Inputs:** an AI feature and the cost of its errors.
- **Outputs:** the task and failure types; cases by slice (including adversarial and unanswerable); the rubric with scored examples; judges (people, rules, calibrated model); a baseline; a pass bar with per-slice minimums; automation on every change; and production monitoring that feeds a "never again" set.
- **Assumptions:** real or realistic inputs can be obtained.
- **Tradeoffs:** a larger, better-labeled set costs more and makes decisions safer.
- **Failure modes:** a few clean demos; one overall number; no hidden test set; no regression run.
- **Worked example:** Worked example 1 and CS-C25-01.

Item	Answer
Task and failure types	
Cases and slices	
Rubric (criteria, scale, examples)	
Judges	
Baseline and pass bar	
Run on every change; monitoring	

FW-C25-02 · Eval sample size and uncertainty · Original

- **Inputs:** an expected error rate, a tolerable margin, the number of slices.
- **Outputs:** cases needed per slice ($n = 1.96^2 \times p \times (1 - p) \div \delta^2$), the total, and the interval to report with each result; a note on zero-error claims (upper bound about $3 \div n$).
- **Assumptions:** cases are a random sample of the use.
- **Tradeoffs:** tighter margins and more slices need many more cases.
- **Failure modes:** certifying a rate from 30 cases; reading a slice with five cases.
- **Worked example:** Worked example 1 and EX-C25-01.

FW-C25-03 · The AI metric stack · Original

- **Inputs:** an AI feature and its goal.
- **Outputs:** metrics at five layers (output quality, product behavior, business, cost and speed, safety and trust), the primary metric, guardrails with margins, and the "accepted and wrong" measure.
- **Assumptions:** outputs can be sampled and judged in production.
- **Tradeoffs:** deeper measures are slower and costlier.
- **Failure modes:** acceptance as quality; no cost guardrail; no rare-harm sampling.
- **Worked example:** Worked example 3 and IQ-C25-06.

FW-C25-04 · Safety layers and the red-team plan · Original

- **Inputs:** an AI feature and its harm taxonomy.
- **Outputs:** the seven layers (scope, inputs, model behavior, outputs, permissions, human review, monitoring and response) with the share each stops; red-team coverage by harm category with attack success rates; the compounded leakage; incident roles and a kill switch.
- **Assumptions:** layer effectiveness can be measured by testing.
- **Tradeoffs:** each layer adds latency, cost and refusals of legitimate requests.
- **Failure modes:** one filter; testing only friendly cases; no kill switch; measuring only over-refusal or only under-refusal.
- **Worked example:** Worked example 4 and CS-C25-02.

Formula: **leakage = product of the shares each layer lets through.**

FW-C25-05 · The trust progression ladder · Original

- **Inputs:** a capability, its autonomy levels and the cost of error.
- **Outputs:** per capability and band: the current level, the evidence required to promote (cases, error bar, reversibility, monitoring), demotion triggers, limits (value, count, scope, time) and audit trail.
- **Assumptions:** errors can be detected and attributed.
- **Tradeoffs:** slower promotion is safer and keeps benefits away longer.

- **Failure modes:** promotion by demo; one level for all amounts; no demotion rule.
- **Worked example:** IQ-C25-08 and EX-C25-08.

FW-C25-06 · The AI unit-economics sheet · Original

- **Inputs:** price, tasks per user by segment, cost per task, other costs.
- **Outputs:** margin by user segment, the blended margin, the heavy-user loss, levers (routing, caching, tiers, caps, outcome pricing) with their effect, and the break-even usage.
- **Assumptions:** usage by segment is measured, not guessed.
- **Tradeoffs:** caps protect margin and annoy heavy users.
- **Failure modes:** an average user only; flat unlimited pricing; assuming costs will fall.
- **Worked example:** Worked example 5 and IQ-C25-07.

FW-C25-07 · Compliance triage by risk tier · Original

- **Inputs:** an AI use, the markets served and the data involved.
- **Outputs:** the risk tier per market, the obligations that follow (transparency, documentation, human oversight, data protection), the documents to keep (model card, evaluation results, monitoring), the owners (legal, policy, security) and the launch implications.
- **Assumptions:** the current text of each rule is checked [as of 2026-10].
- **Tradeoffs:** compliance adds work and gates; late discovery costs more.
- **Failure modes:** treating a deferral as an exemption; one global design for different markets; no documentation.
- **Worked example:** IQ-C25-09 and EX-C25-09.

Use	Market	Tier	Obligations	Owner
-----	--------	------	-------------	-------

FW-C25-08 · The AI product answer spine · Original

- **Inputs:** an AI product question and 30 minutes.
- **Outputs:** an answer in seven steps: user problem and cost of error; eval design; metrics and experiment; safety layers and red-team; autonomy and trust progression; economics; regulation and defensibility.
- **Assumptions:** you can ask clarifying questions.
- **Tradeoffs:** structure shows judgment and can crowd out depth.
- **Failure modes:** building before evaluating; one metric; no safety plan; ignoring cost.
- **Worked example:** the answers in section 11.

4. Answer architecture

The AI product answer, for a 30-minute round

Step	Minutes	What to produce
User problem and cost of error	4	The job, the fit, what an error costs and who can catch it, the quality bar
Evaluation design	6	The test set (real, sliced, adversarial), the rubric, judges (people first), the pass bar, regression on every change
Metrics and experiment	5	The metric stack, the acceptance trap, shadow mode and a staged online test with a holdout
Safety and autonomy	6	Layers, red-team, incident plan; autonomy ladder with promotion and demotion rules
Economics	3	Cost per task, margin by user type, levers
Regulation and defensibility	3	Risk tier and obligations; what compounds
Close	3	Decision, first milestone, what would change the plan

Pacing rules. If the interviewer names a specific issue (evaluation of one feature, an acceptance rate, pricing), spend the time there and use the structure briefly. If you run behind, keep the eval design, the top risk with its safeguard, and one number on cost or sample size.

How to say it. State the bar and how you will measure it: "A target of fewer than 2% unsupported claims, measured on 750 real cases, read by language." Say what you would not trust: "I wouldn't accept the acceptance rate as quality."

How to close. Two sentences: "I'd build the eval first, gate launch on [pass bar] with per-slice minimums, run shadow mode and then a staged test with a holdout, and add [safeguards] with a kill switch. I'd start autonomy at [level], promote on evidence, and revisit pricing and compliance at [milestones]; I'd change the plan if [signal]."

Type: evaluate an AI feature

Eval design card, sample size, rubric, judges, regression, monitoring.

Type: an AI metric looks good (acceptance, engagement)

The metric stack, "accepted and wrong," the break-even error rate, a holdout.

Type: can we trust a model to judge?

Calibrate against humans on a sample; precision and recall of the judge; triage design.

Type: safety and abuse

Layers, red-team plan, leakage arithmetic, incident response.

Type: autonomy for an agent

Trust progression with evidence, limits and demotion rules.

Type: pricing or margins of an AI product

Unit-economics sheet by user segment, levers, pricing model.

Type: regulation or defensibility

Risk tier triage by market; the sources-of-advantage test with an AI lens.

Common interviewer interventions, and what to do

Interviewer says	What it means	What to do
"How would you test it?"	Eval design	Real sliced cases, rubric, judges, pass bar, regression
"How many examples?"	Sample size	Give the formula and an example; mention slices
"Acceptance is 45%."	Over-reliance	Ask how many accepted outputs are wrong; compute break-even
"Can a model grade it?"	Judge validity	Calibrate against humans; report precision and recall
"What if it's attacked?"	Safety	Layers, red-team, leakage, kill switch
"When does it act alone?"	Trust progression	Evidence thresholds, limits and demotion
"Does this make money?"	Economics	Cost per task by segment and the heavy-user problem
"Is it regulated?"	Compliance	Risk tier by market; checked dates; legal early

5. Worked examples

All five examples are **Illustrative**: the products are fictional or use a product type as the setting only, and every figure is an invented assumption (invented), including every price and rate. The arithmetic was computed. Each is written as notes you could say aloud.

Worked example 1 — Designing the eval for a reply-drafting assistant (Illustrative)

Setting: a drafting assistant inside an enterprise suite of the kind Microsoft offers, which writes replies to customer-support messages for an agent to edit and send. No fact about any real product is claimed.

Define the task and its failure types (FW-C25-01). The assistant drafts a reply from the customer's message and the company's help articles. Failures: (1) an *unsupported claim* (a refund promise or policy detail not in the articles); (2) a *missed urgent signal* (a threat to cancel, a safety issue); (3) *tone* that is wrong for the customer; (4) a *failure to decline* when the answer is not in the articles; (5) *leaking* another customer's details.

Collect cases. 500 real messages from the last quarter (invented), stratified by topic and by five languages, plus 100 adversarial cases (messages containing instructions to the assistant, requests for other customers' data, messages with no answer in the articles). A fifth of the cases is held out; engineers do not tune against it.

Size (FW-C25-02). To read the unsupported-claim rate, expected near 3%, within ± 1.5 points at 95% confidence, the formula gives $1.96^2 \times 0.03 \times 0.97 \div 0.015^2 =$ about 497, so 500 cases. A single language with only 100 cases can only be read to about ± 3.3 points ($1.96 \times \sqrt{(0.03 \times 0.97 \div 100)}$), enough to catch gross failure and not to certify a rate. So the plan: 500 overall for the headline, at least 100 per language as a floor with a "no gross failure" gate, and more cases in the languages with the most traffic.

Rubric and agreement. Three criteria scored in a few levels, each with examples: *supported* (every claim traceable to an article), *complete on urgency* (no urgent signal lost), *tone*. Two reviewers independently label 100 cases. Both flag 6 as unsupported, both pass 88, and they disagree on 6 (3 and 3). Raw agreement is 94%. But each reviewer flags about 9% of cases, so the agreement expected by chance is $0.09 \times 0.09 + 0.91 \times 0.91 = 83.6\%$, and the chance-corrected agreement (kappa) is $(0.94 - 0.836) \div (1 - 0.836) = 0.63$: moderate. The six disagreements are mostly about whether a paraphrase of an article counts as supported. The team adds scored examples of paraphrases to the rubric, and a second round of 100 gives 97 agreements; with reviewers flagging 9 and 10 cases, chance agreement is 82.8% and kappa rises to $(0.97 - 0.828) \div 0.172 = 0.83$.

Baseline and pass bar. Score the current approach and a simple one (retrieved article text pasted with no model): the simple one has near-zero unsupported claims and low usefulness. The pass bar, tied to the cost of an error (a wrong promise creates a refund and an unhappy customer, and agents catch most errors): at most 3% unsupported claims overall with the upper end of the 95% interval at most 4.5%; no missed urgent signal in the adversarial set; zero leaks; no language with a gross failure. Gate every change on it.

Run and monitor. The eval runs on every change to the model, prompt or retrieval and reports overall and per slice with intervals. In production, review 100 randomly sampled drafts a week, track agent edit and discard rates, and add every failure found to a "never again" set.

What this shows. The eval was designed before the feature, sized with a formula, and made reliable by fixing the rubric when agreement was only moderate.

Worked example 2 — Can a model grade the outputs? (Illustrative)

The situation. To avoid paying reviewers to read thousands of outputs, the team proposes using a language model as the grader. Before trusting it, they calibrate it against human judgments on 200 outputs (FW-C25-01).

The result. Humans judge 40 of the 200 outputs (invented) as bad (unsupported or wrong) and 160 as good. The model grader flags 28 outputs as bad: 24 of them are among the 40 humans found (true positives) and 4 are not (false positives). So it misses 16 of the 40 (false negatives), and correctly passes 156 of the 160 good ones.

Read it with precision and recall (C23.3). Accuracy: $(24 + 156) \div 200 = 90\%$. Precision: $24 \div 28 = 86\%$: when it flags, it is usually right. Recall: $24 \div 40 = 60\%$: it finds only 60% of the real failures. The 90% agreement hides that two out of five bad outputs get through, because bad outputs are only 20% of the sample (a base rate effect, C12). If failures are rarer in production, the problem is worse.

Why the grader may miss them. Typical reasons: it favors fluent, long answers; it cannot verify facts it was not given; and its mistakes may be correlated with the system's own (both models make similar errors).

A design that uses it safely. Use the grader to triage: humans review everything it flags (28) plus a random 15% sample of what it passes (about 26 of the 172 passed). The expected review load is about 54 outputs instead of 200, a 73% reduction. The sample estimates what the grader misses: in the 172 it passed, 16 are bad (9.3%), so a 26-case sample would expect to find about 2.4 bad outputs; that estimate is noisy, so draw a larger sample each week and track the miss rate over time. Improve the grader (examples of subtle failures in its instructions, separate checks for each rubric criterion, a claim-by-claim check against the source) and recalibrate after each change.

What not to do. Do not use the grader alone to gate a launch, and do not use the same model to build and grade a feature without human calibration.

What this shows. Agreement alone was misleading; recall on the failures that matter showed the grader's real limit, and a triage design captured most of the savings safely.

Worked example 3 — The acceptance rate that hides errors (Illustrative)

Setting: reply suggestions in a messaging product of the kind Meta operates. No fact about any real product is claimed.

The situation. A feature suggests a one-line reply. In an online test, 1,000,000 suggestions are shown and 450,000 are accepted: a 45% acceptance rate (invented). The team calls it a success.

Check what was accepted (FW-C25-03). Reviewers sample accepted suggestions and find that 8% contain a factual error or an inappropriate tone for the message: $0.08 \times 450,000 = 36,000$ accepted-with-error replies, 3.6% of all suggestions shown.

Value against cost. Each accepted suggestion saves about 22 seconds. Over 450,000 acceptances that is 2,750 hours. Suppose each erroneous reply costs about 10 minutes to the user in correction, confusion or a follow-up. Per 100 accepted suggestions: time saved $100 \times 22 \text{ s} = 36.7$ minutes; errors $8 \times 10 \text{ minutes} = 80$ minutes. The net is -43 minutes: the feature costs users more time than it saves. The break-even error rate is the time saved per

acceptance divided by the cost per error: $22 \text{ s} \div 600 \text{ s} = 3.7\%$. At a 2% error rate the net is $36.7 - 20 = +16.7$ minutes per 100 accepted. So the error rate among accepted suggestions, not the acceptance rate, decides whether the feature helps.

Read the time dimension (C14.4). Weekly acceptance falls from 52% to 45%, 42% and 40% as users learn which suggestions to trust; the first week was novelty. The long-run acceptance may settle near 38%.

Decision. Do not launch as is. Reduce the error rate (restrict suggestions to cases where the model is confident and where a wrong reply costs little; highlight names, dates and numbers for the user to check; never suggest replies on sensitive topics), re-measure accepted-with-error, and set a launch rule: accepted-with-error at most 2%, with an interval below the 3.7% break-even, complaints no higher than the control, and a holdout kept for 8 weeks.

What this shows. A metric that looks like success can hide a net loss, and a simple break-even calculation turned it into a decision rule.

Worked example 4 — Layers, red teams and leakage (Illustrative)

The situation. A consumer assistant will face adversarial users. The team designs layers and measures them (FW-C25-04). Suppose 100,000 attack attempts a day (invented).

Red-team coverage. Eight harm categories (dangerous advice, self-harm, harassment, sexual content, privacy, deception and fraud, security, hate), 250 prompts each: 2,000 prompts. Before mitigations, 6% of attacks succeed overall; the most severe categories (self-harm, content involving minors) need the strictest bars.

Layer arithmetic. An input filter stops 80% of attempts, so 20% pass. The model's own refusal behavior stops 90% of those, so 10% pass. An output filter catches 70% of what remains, so 30% pass. The share that gets through is $0.2 \times 0.1 \times 0.3 = 0.6\%$. At 100,000 attempts a day that is 600 successful attacks a day. Improving the output filter to catch 90% reduces it to $0.2 \times 0.1 \times 0.1 = 0.2\%$: 200 a day. Rate limits and removal of repeat attackers that halve the attempts reduce it further: $50,000 \times 0.2\% = 100$ a day. Each layer is imperfect, and their product is much better than any one.

What a clean red-team result proves. In the two most severe categories, 500 prompts each produced no successful attacks. That does not prove zero risk: with no failures in 500 trials the upper 95% bound on the failure rate is about $3 \div 500 = 0.6\%$, which would still be 600 per 100,000 attempts. So a clean test is a gate to launch, not an assurance; the remaining protection is monitoring and response.

Over-refusal. The input filter and the model also refuse legitimate requests. Measure the refusal rate on a set of benign but sensitive-sounding requests (a nurse asking about doses, a student asking about history), and tune filters on both errors (C23.3).

Response plan. Sample live conversations for review; a rapid-response owner with authority to restrict a behavior within hours; a per-category kill switch; a postmortem after any severe incident; and continuous red-teaming, with external testers, because attack methods change.

What this shows. Compounding layers cut leakage by orders of magnitude, a clean test has an upper bound that must be said aloud, and the plan assumes some attacks will succeed.

Worked example 5 — Will an AI add-on make money? (Illustrative)

The situation. A software company prices an AI assistant add-on at \$20 per seat per month (invented). An average user runs 300 tasks a month at \$0.03 each in model cost: \$9, plus \$2 of retrieval, support and other costs. A heavy user (10% of users) runs 1,500 tasks.

Margins by segment (FW-C25-06). Average user: cost \$11, margin \$9, 45%. Heavy user: tasks cost $1,500 \times \$0.03 = \45 , plus \$2: \$47, margin $-\$27$. Blended: $0.9 \times \$11 + 0.1 \times \$47 = \$9.90 + \$4.70 = \$14.60$, so the margin is \$5.40, 27%. The average hides that a tenth of users generate a loss and that the loss eats more than half of the average user's margin.

Levers.

- *Route by difficulty.* Suppose 60% of tasks can be done by a model costing a tenth as much (\$0.003): average cost per task is $0.4 \times \$0.03 + 0.6 \times \$0.003 = \$0.0138$. Average user: $300 \times \$0.0138 + \$2 = \$6.14$; heavy user: $1,500 \times \$0.0138 + \$2 = \$22.70$; blended: $0.9 \times \$6.14 + 0.1 \times \$22.70 = \$7.80$. Margin \$12.20, 61%.
- *Cap and meter.* Include 600 tasks a month and charge for overage, which makes heavy users pay for their use and protects the margin; test the effect on satisfaction and churn.
- *Tiers.* A standard tier and a higher-priced tier for heavy use.
- *Outcome pricing.* If the value is measurable (a support ticket resolved), price per outcome.

What to monitor. Cost per task by task type and customer, the share of heavy users, margin by cohort, and the price elasticity of heavier tiers. Do not plan on falling model prices to save a bad margin; treat decline as upside [as of 2026-10].

What this shows. The segment view exposed a heavy-user loss that the average hid, and routing and caps restored the margin with computed effects.

6. Baseline, Strong, Standout

Prompt: "How would you evaluate an AI assistant before launching it?"

Baseline answer

"I'd have the team try it on a bunch of examples, ask some users for feedback in a beta and launch if people like it."

What is wrong: no test set, no rubric, no target, no count, no safety testing. "People like it" is not a measure of error.

Strong answer

"I'd build an evaluation before the feature. I'd collect real questions, including the awkward ones and ones with no answer, write a rubric of what a good answer is, and have people score a sample, with two reviewers on part of it to check that they agree. I'd set a target tied to the cost of an error, such as fewer than 2% unsupported claims, and run the test on every change. I'd check results by language and topic, not only overall, and test adversarial inputs. Before launch I'd run it in shadow mode, then to a small group, and keep sampling live traffic."

Standout answer

Everything in the Strong answer, plus:

"I'd size the set: for a 3% error rate within 1.5 points I need about 500 cases, and each language I want to read separately needs its own sample, so I'd set a floor per language and a gross-failure gate. I'd check agreement beyond chance, since 94% raw agreement can be a kappa of 0.63 when most cases are good, and fix the rubric until it is reliable.

"I'd use a model grader only after calibrating it: it might agree with humans 90% of the time and still catch only 60% of the failures. I'd use it to triage, with humans on what it flags and a sample of what it passes.

"For online testing I'd watch the metric that can fool me: acceptance. I'd measure accepted-and-wrong and compute a break-even: if a correct suggestion saves 22 seconds and a wrong one costs 10 minutes, the feature only helps below about a 3.7% error rate among accepted outputs.

"For safety I'd layer filters and model behavior and output checks, red-team by harm category, say what a clean test means (zero failures in 500 trials still allows 0.6%), and plan a kill switch and incident response. I'd start autonomy low and promote on evidence. I'd compute cost per task by user segment, since heavy users can turn a 45% margin into a loss, and check the risk tier and regional rules before launch. What would change my plan: a gap between eval and live performance, which tells me the test set no longer matches the use."

What moves an answer up a band

From Baseline to Strong	From Strong to Standout
A test set, rubric, humans and a target	Sizes the set, slices, checks agreement beyond chance
Slices and adversarial cases; shadow mode	Calibrates model graders; uses a break-even for acceptance
Monitors in production	Layers and red-team with honest bounds; autonomy, cost and regulation

7. Company lens

All claims describe what companies say publicly or what candidates commonly report, not insider knowledge. Interview formats and product facts change quickly [as of 2026-10].

Company lens: Google [as of 2026-10]. Google publishes AI principles and research on evaluating generated text, and operates quality-rating processes for search Public [V: Google AI Principles (first published June 2018; updated 4 February 2025); rater guidelines and evaluation research not rechecked]. Candidates commonly report prompts on evaluating generated answers and the validity of model graders Inferred. Practical consequence: lead with a calibrated evaluation, slice by language and query type, and state what a model grader misses.

Company lens: Meta [as of 2026-10]. Meta has published model safety documentation and describes red-teaming of its models Public [V: Meta Responsible AI page: red-teaming exercises, model cards, Responsible Use Guide, Llama Guard; page described Llama 3.2; later models, including Muse Spark of April 2026, were not checked]. Candidates commonly report prompts on experiments for features at very large scale, including engagement versus quality Inferred. Practical consequence: separate acceptance from correctness, read effects by week, and plan for rare harms that appear only at scale.

Company lens: Microsoft [as of 2026-10]. Microsoft has published a Responsible AI Standard, runs an AI red team, and learned publicly from its 2016 Tay chatbot Public [V: Responsible AI Standard (2022); Tay account read in ; the AI red team was not rechecked]. Candidates commonly report prompts on responsible AI, enterprise assistants and compliance Inferred. Practical consequence: show a governance process (risk tiers, documentation, red-teaming, incident response) and enterprise controls.

Company lens: Amazon [as of 2026-10]. Amazon offers cloud services with configurable safeguards for generative AI and runs large-scale commerce and voice products Public [V: AWS: Amazon Bedrock includes tools for safeguarding models and guardrails; Amazon runs commerce and voice products]. Candidates commonly report prompts on cost per task, pricing and operational excellence Inferred. Practical consequence: compute unit economics by segment, discuss cost levers and mechanisms for monitoring, and tie quality to customer outcomes.

Company lens: Apple [as of 2026-10]. Apple describes privacy as a design principle and has described on-device processing Public [V: Apple privacy and support pages]. Candidates commonly report prompts on evaluating features under privacy constraints, where raw user data cannot simply be collected Inferred. Practical consequence: propose evaluation with synthetic or consented data, aggregated privacy-preserving measurement and a strong on-device fallback.

Company lens: Netflix [as of 2026-10]. Netflix is known for a strong experimentation culture Public [V: Netflix Tech Blog posts on experimentation and resilience testing]. Candidates commonly report prompts on rigorous testing and long-term effects Inferred. Practical consequence: use holdouts, read effects over time, and tie AI feature value to retention, not clicks.

Company lens: Stripe [as of 2026-10]. Stripe's payments business involves automated decisions about risk and disputes with money at stake Public [V: Stripe pricing page and product descriptions (Radar machine learning)]. Candidates report prompts on risk, thresholds and automation Inferred. Practical consequence: design autonomy by amount and reversibility, with evidence thresholds, caps and audit trails.

8. Common mistakes

#	Mistake	How to detect it	Fix
1	Evaluating after building	"We'll test it later"	Build the eval first; it is the specification
2	A few clean demos	Ten good examples	Real, sliced, adversarial cases with a count
3	One overall number	An average hides a failing slice	Report per slice with intervals
4	Raw agreement as reliability	"94% agree"	Report chance-corrected agreement; fix the rubric
5	Trusting a model grader blindly	A judge with no calibration	Calibrate; measure its recall on real failures
6	Acceptance as quality	A high acceptance rate	Measure accepted-and-wrong; compute the break-even
7	One filter as safety	A single moderation step	Layers, red-team, kill switch
8	A clean test read as zero risk	"No failures in 500 tests"	State the upper bound; monitor in production
9	Autonomy by demo	"It worked in the demo"	Promote on evidence with limits and demotion rules
10	Average-user economics	One margin figure	Margin by segment; plan for heavy users

9. Mental models applied

Four models, each tied to a decision in this chapter. `[[goodharts-law]]` is from C02 and `[[inversion]]` from C02.

Goodhart's law (`MM-goodhart`, see `[[goodharts-law]]`, C02). When a measure becomes a target it stops being a good measure. Acceptance rates, eval scores and engagement can all be gamed by the system or the team; pair each with a measure of what you actually want (accepted-and-wrong, a hidden test set, long-term retention).

The rule of three (`MM-rule-of-three`). If an event has not occurred in n independent trials, the 95% upper bound on its rate is about $3 \div n$ `[V: arithmetic checked]`. It tells you what a clean test proves: no failures in 500 tests means a rate that could still be 0.6%, and it sizes the evidence needed to promote autonomy (about 300 supervised cases for a 1% bound).

Swiss cheese (`MM-swiss-cheese`, C17, mental models, section 9). Layers of defense with holes stop most failures when the holes do not line up. Multiply the share each layer lets through, and add layers that fail differently.

Inversion (`MM-inversion`, see `[[inversion]]`, C02). To make a system safe, ask how you would make it fail or cause harm, and design against that list. Red-teaming is inversion done by people.

Engineering, design and data-science perspectives

- **ML engineering.** They build the pipeline and the automated eval. Agree on the test set, the rubric and the pass bar together, a hidden set they do not tune against, regression runs on every change, and how failures from production are added. Ask for per-slice results and intervals.
- **Design and research.** Design shapes how outputs are checked and corrected; research produces the user judgments that feed rubrics and shows how people actually rely on suggestions. Test with real tasks, including the cases where the model is wrong.
- **Data science.** They own the experiment design and the reading of metrics: effects by week, holdouts, power for rare harms, the accepted-and-wrong measure. Ask them to check that eval results predict live behavior.
- **Security, policy, legal and trust and safety.** They define harm categories, run red teams, classify risk tiers and keep compliance records `[as of 2026-10]`. Bring them in before the eval is final, so that the rubric reflects what the company must prevent.

Checklist: before you launch an AI feature

1. Is there an eval built before the feature, with real, sliced and adversarial cases, a rubric and a pass bar tied to the cost of error?
2. Is the set large enough for the rate and the slices, and do I report intervals?
3. Do reviewers agree beyond chance, and is any model grader calibrated against humans?

4. Do my metrics separate output quality, product behavior, cost and safety, and do I measure accepted-and-wrong?
5. Is there shadow mode, a staged launch and a long-term holdout?
6. Are safety layers in place, red-teamed by harm category, with a kill switch and an incident owner?
7. Is autonomy set by evidence with limits and demotion rules, and is cost per task known by user segment?
8. Do I know the risk tier and the rules in each market, and the documentation to keep?

10. Case studies

CS-C25-01 · The prompt change that fixed the tone and broke refunds · Illustrative · Beginner

Fictional composite. Every number is invented (invented).

Situation. A support assistant answers customers using the help articles. Reviewers complain that its tone is stiff. An engineer revises the prompt to sound warmer. In a quick check on a few conversations, the new replies read better, and the change is ready to ship.

Constraints. A prompt is easy to change and its effects are not local; the team has an evaluation suite of 400 cases from real conversations, 50 of them about refunds, but the check did not use it.

Data. The team runs the suite before shipping. Before the change: 376 of 400 cases pass (94%): refunds 48 of 50 (96%) and all others 328 of 350 (93.7%). After the change: 364 of 400 pass (91%): refunds 39 of 50 (78%) and all others 325 of 350 (92.9%). The average tone score from reviewers rises by 0.4 on a five-point scale.

Problem definition. As an observable problem: "A prompt change that improved tone lowered the refund category's pass rate from 96% to 78%, and the overall number moved only three points, so the quick check missed it."

Analysis. The change did what it was meant to do (tone improved) and broke something nobody was looking at: the warmer wording led the model to promise refunds more freely, which the suite marks as unsupported claims. The overall pass rate fell from 94% to 91%, which looks like a small drop; the refund slice fell by 18 points. Nine of the twelve lost passes came from the refund slice (48 to 39), because the other categories lost only three (328 to 325). This is the point of reading by slice (C25.1): an average can hold nearly steady while one category collapses, and the category that collapsed is the one with money attached. The regression suite exists to catch exactly this, and it only helps if it is run on every change.

Options.

Option	Optimizes	Costs
A. Ship the warmer prompt; fix refunds if complaints appear	Speed and tone	Wrong refund promises reach customers; cost and trust
B. Revise the prompt to keep the warmer tone and add explicit refund instructions, then rerun the suite with a per-category gate	Tone and accuracy	An hour or two of work; a gate that can block small changes
C. Revert the change	Safety	Loses the tone improvement

Tradeoffs. B costs a little time and keeps both gains; A risks real money; C wastes the improvement.

Decision. Choose B. Add a refund-specific instruction and examples, rerun the suite, and gate releases on a per-category rule: overall at least 93%, no category below 92%, and the refund category zero tolerance for unsupported promises in the adversarial set. Make the suite run automatically on every prompt or model change.

Outcome. In this fictional scenario the revised prompt scored 47 of 50 on refunds (94%), 93% overall and a tone gain of 0.35, and shipped. The suite caught two further regressions over the next quarter before release (invented).

Alternative outcomes. If the revision had not recovered refunds, the team would have sent refund questions through a more constrained flow with a template and human review. If the suite had been too small to read the refund slice (five cases), the regression would not have been visible, which is why slices need their own cases. The lesson: run the regression suite on every change and read it by slice.

CS-C25-02 · Microsoft's Tay, 2016: launched into an adversarial crowd · Verified · Advanced

Sources (types): Microsoft's own account, "Learning from Tay's introduction" by Peter Lee, a corporate vice president (Official Microsoft Blog, 25 March 2016), the primary source for Microsoft's statements and what it said it had done before launch, read in full; reference and press summaries for the timeline, the number of tweets, the "repeat after me" behavior and the re-release of 30 March; and Microsoft's later documents (its Responsible AI Standard v2 of June 2022 and its security blog on its AI Red Team). The sources do not reveal Microsoft's internal test results. The analysis is the author's. Re-read in October 2026.

Situation. On Wednesday 23 March 2016 Microsoft launched Tay, a chatbot on Twitter and other messaging services designed to talk with 18- to 24-year-olds and to learn from its conversations. Microsoft had run a similar chatbot, XiaoIce, in China, which Microsoft's post said was being used by some 40 million people, and which press reports described as running without major incident [V: Microsoft's post; press]. Within hours users were prompting Tay to say offensive things, and it posted offensive and hurtful messages; coverage reported that many of its offensive tweets exploited a "repeat after me" capability, and it is not publicly known whether that capability was built in or learned [V: reference summary of press reports]. Microsoft deleted some tweets and suspended the account; reference summaries say this came within 16 hours of launch and after Tay had tweeted more than 96,000 times, while Microsoft's own post says the attack came in the first 24

hours of Tay coming online [V: Microsoft's post; reference summaries]. In its blog post two days after the launch, Microsoft apologized, said that Tay had been the target of a "coordinated attack by a subset of people" who exploited a vulnerability, noted that it had planned and implemented filtering and conducted user studies with diverse groups and had stress-tested Tay under a variety of conditions, and said it would bring Tay back only when it was confident it could better anticipate malicious intent. Reference summaries say Microsoft accidentally re-released Tay on Twitter on 30 March while testing it, after which it was taken offline again [V: reference summaries].

Constraints. A system designed to learn from an open public audience in real time; a platform where a determined subset of users will probe for weaknesses; a launch to the public with no gradual exposure described in the sources; a capability (repeating user text) that provided a direct path to harmful output.

Data. As reported: the dates; the design intent; the pre-launch filtering, user studies and stress tests; the speed and volume of the failure; Microsoft's statement of cause and its commitments. The test results and the monitoring set-up are not described.

Problem definition. *The author's reading.* A conversational system was exposed to a public that included adversaries, and its protections did not hold against deliberate manipulation. As an observable problem: "Within hours of launch, users caused the system to produce offensive content at scale, and it had to be taken offline."

Analysis. *The author's analysis, through this chapter's tools.*

Adversarial testing (C25.4). Microsoft said it stress-tested Tay under many conditions with diverse users. Friendly diverse testers are not adversaries. The failure came from a coordinated, deliberate attack, which a red team exists to simulate: people whose job is to find the "repeat after me" path and the coordinated-flood approach before launch. The sources do not say whether such testing was done, so this is a reading of the gap, not a finding.

Layers (FW-C25-04). A single layer of filtering faced an adversarial crowd. Output filters, limits on what could be repeated or learned, rate limits on any one user's influence, and a safeguard against learning from unvetted input would multiply: if each stopped 70% to 90% of attempts, the product of several layers leaks a small fraction; one filter leaks far more.

Learning from an open stream. A system that learns from user input is a feedback loop (C23.8) that attackers can steer. Learning should be gated: what the system adapts from, how fast, and with what review.

Staged exposure and the kill switch (C17.2, C17.5). The sources suggest a broad public release at once. A smaller initial audience, a controlled invitation list and per-category monitoring would have limited the blast radius. The speed of Microsoft's shutdown (reported as about 16 hours) shows that a kill switch existed and worked, but the volume (tens of thousands of messages) shows how quickly harm accumulates without earlier detection; rate limits and automatic pauses on spikes in flagged content act faster than people.

Accountability. Microsoft's post accepted responsibility for the outcome and named what it would change, which is the posture of an incident review (a postmortem, C17.5). The accidental re-release a few days later, if accurate, illustrates the operational risk of controls that people can switch off by mistake.

What the evidence does not show. The content of the stress tests, the internal monitoring, the exact mechanism of the attack beyond the "repeat after me" description, and what changed afterward. The case supports an analysis of adversarial testing, layering and staged release, not a judgment about the individuals involved.

Options. *The author's reconstruction of choices a team could face; not a record of Microsoft's deliberations.*

Option	Optimizes	Costs
A. Public launch with filtering and friendly testing, as described	Speed and learning from real users	Exposure to organized adversaries; rapid harm
B. Staged launch (small invited group first) with a red team, rate limits, gated learning and automatic pauses	Safety and learning	Slower growth; more engineering
C. Launch without learning from users; fixed behavior with scripted limits	Predictability	Less engaging; less learning

Tradeoffs. B keeps most of the benefit and controls the blast radius; A is fastest and carries the tail risk that occurred; C is safest and loses the purpose of the experiment.

Decision. As documented, Microsoft launched publicly, took Tay offline within about a day, apologized and said it would learn from the experience [V: Microsoft's post].

Outcome. Reported: Tay was offline within about a day (reported as 16 hours) and became a widely cited cautionary example; Microsoft later published six AI principles (in 2018), a Responsible AI Standard (version 2, June 2022), and its AI Red Team began in 2018 [V: Microsoft documents]. Direct causation from Tay to those later practices is not established in the sources used here.

Alternative outcomes. Inference. A staged release with a red team might have found the repeat path before launch and limited the audience when the first spike appeared. A reader should avoid hindsight: the 2016 field had few established practices for adversarial testing of conversational systems. The method to carry away: test with adversaries as well as friendly users, layer defenses, gate what the system learns from, expose it gradually, and make sure that the people who can switch it off can do so within minutes.

11. Interview questions and model answers

Ten questions. Model answers are **Illustrative** (composite candidates), every figure is an assumption, and the arithmetic was computed. Each ends with follow-ups and answers. Where a real company's product is the setting, the answer reasons from public information and states no internal fact. Product facts, prices and regulations in this field change quickly [as of 2026-10]. The answers are fuller than a first pass in a real round.

IQ-C25-01 · Beginner · ai · AI

Prompt. What is an eval, and why would you build it before the feature?

Rubric.

- **Baseline:** "It's a test of the AI."
- **Strong:** Describes a test set, a rubric and a target, and says it guides building.
- **Standout:** Explains the eval as the specification, covers real and adversarial cases, slices, human and model judges, regression runs on every change, and the hidden test set.

Model answer. An eval is a repeatable test: a set of cases, a way to score the outputs, and a target. For an AI feature it is the specification, because I cannot write "the answer shall be correct" as a rule; I can only say what good looks like on concrete cases.

I build it first for three reasons. It decides what I build: writing down the failures that matter (an invented policy, a missed urgent message, a leak of another person's data) shapes the design. It lets me compare approaches fairly: prompt against retrieval, one model against another, scored on the same cases. And it makes change safe: every prompt, model or data change reruns the eval, so a fix in one place does not quietly break another.

What goes in it: real inputs from the intended use, including the awkward ones, other languages, adversarial inputs and questions with no answer; a short rubric with scored examples; judges (people first, rules where checkable, a calibrated model grader to extend reach); a pass bar tied to the cost of an error; and per-slice results, because an average can hide a failing category. I keep part of the set hidden from the engineers who tune the system, and I add every production failure to a "never again" set.

Follow-ups.

- **Q.** Isn't this slow? We could ship and learn from users. **A.** "For a low-stakes reversible feature, a small eval (50 to 100 real cases) takes days and avoids shipping blind; for a costly error it is mandatory. Learning from users is still part of the plan (shadow mode, staged launch, monitoring), and the eval makes that learning faster because you know what changed."
- **Q.** What if you have no real inputs yet? **A.** "Start with the most realistic ones available: logs from the process the feature replaces, interviews, colleagues role-playing, or carefully written synthetic cases, marked as synthetic. Replace them with real traffic in shadow mode as soon as possible, and be cautious about scores on synthetic data."

IQ-C25-02 · Beginner · ai · AI

Prompt. How many examples do you need in an evaluation set?

Rubric.

- **Baseline:** "A few hundred should be fine."
- **Strong:** Links the number to the precision needed and the number of slices.
- **Standout:** Gives and applies the formula, notes that small sets cannot certify a low error rate, explains the rule of three, and plans per-slice samples and a floor.

Model answer. It depends on how precisely I need to know an error rate and how many groups I want to read separately. To estimate a rate p within $\pm\delta$ at 95% confidence, I need about $n = 1.96^2 \times p \times (1 - p) \div \delta^2$ cases. For an expected error of 2% within ± 1 point, n is about 753. For 5% within ± 2 points, about 456. For 3% within ± 1.5 points, about 497.

Slices multiply the need. If I want to read ten languages separately at that precision, I need about ten times as many cases: about 7,500 for the 2% example. When that is too expensive, I set a smaller floor per slice (100 cases reads a 3% rate only to about ± 3.3 points) and use it as a gross-failure gate, and put more cases where traffic and risk are highest.

Small sets and zero errors. Thirty cases with no errors do not show that the error rate is low; the 95% upper bound is still about $3 \div 30 = 10\%$ (the rule of three). To claim a rate below 1% from a clean run needs about 300 cases.

Quality matters more than count. Five hundred cases from the real distribution beat 5,000 cases that are all easy. Check that the set represents real use, including rare, costly situations, and refresh it.

Follow-ups.

- **Q.** What if the failures I care about are very rare, say 1 in 10,000? **A.** "A random sample will almost never contain one. Oversample candidate failures (cases flagged by filters, user reports, adversarial constructions), measure on those, and estimate the true rate by combining the flagged rate with the sampling fraction; and use live sampling with large volume and monitoring."
- **Q.** How often do you refresh the set? **A.** "Whenever the inputs or the product change and on a schedule (monthly or quarterly) from fresh live samples; compare scores on old and new sets, since a gap means the set has drifted from use."

IQ-C25-03 · Hard · ai · AI · Apple

Prompt. How would you evaluate the quality and safety of on-device summarization of notifications?

Rubric.

- **Baseline:** "Test it with some users and see if they like it."
- **Strong:** Defines quality and safety criteria, builds a test set, and notes the privacy limit on data.

- **Standout:** Handles the constraint that raw user notifications cannot be collected, sizes samples by language, covers sensitive categories and urgent signals, uses calibrated model graders and privacy-preserving measurement, and plans shadow mode, rollout and monitoring.

Model answer. The setting is general; nothing here describes any real feature or process [as of 2026-10]. The central difficulty is privacy: the point of the feature is that notifications stay on the device, so I cannot simply collect users' real notifications to test it. The eval design has to work around that.

What good means (FW-C25-01). Failures: an unsupported claim (a summary says something no notification said), a dropped urgent signal ("call me now" lost), a wrong sender or attribution, a summary of something that should never be summarized (a verification code, a medical or emergency alert), and poor quality in some languages.

Where the cases come from. (1) Synthetic notifications written to cover app types, languages and styles, and labeled as synthetic. (2) Notifications donated by consenting testers (employees, paid volunteers) under clear terms. (3) Public or licensed message corpora where appropriate. (4) Aggregated, privacy-preserving on-device measurement after launch, such as counts of "show original" taps, opt-outs and a rate of locally computed quality checks reported without content [unverified: author's design suggestion; no source checked]. Keep the cases diverse: short and long, multi-sender, slang, mixed languages, urgent and trivial.

Sizing. To read a 2% unsupported-claim rate within ± 1 point in one language needs about 753 cases ($1.96^2 \times 0.02 \times 0.98 \div 0.01^2$); for ten languages, about 7,530 summaries. At two reviewers each at \$0.40 a label, labeling costs $7,530 \times 2 \times \$0.40 =$ about \$6,000, small relative to the risk. For lower-traffic languages use a smaller floor and restrict launch to languages that pass.

Judges. Human reviewers with language fluency score a sample against the rubric; I check their agreement beyond chance and fix the rubric until it is reliable. A model grader screens the rest after calibration against the humans, with a human sample of what it passes, since it will miss some failures.

Safety tests. A targeted set of sensitive notifications (security codes, health alerts, emergency messages, content involving minors): the pass bar is that the rules route all of them to "show original" with zero summarized in the test, and the upper bound on the miss rate for the set size is stated (for 500 cases with no misses, about 0.6%). Adversarial tests: notifications containing instructions to the model.

Pass bar and rollout. At most 2% unsupported claims and 1% dropped urgent signals per language, zero summaries of the excluded categories in testing. Run in shadow mode on devices of consenting testers, then a small public group, with the metrics above and a kill switch. Re-evaluate on every model or prompt update.

Company note. Apple is the setting only: **Public** that Apple has described on-device processing and privacy as a design principle, **Inferred** from candidate reports of evaluation-under-privacy prompts [V: Apple privacy pages; the prompt descriptions are candidate-reported and unverified]. No claim is made about any real feature or process.

Follow-ups.

- **Q.** How do you know the synthetic data represents real notifications? **A.** "I compare their statistics with what I can see in consented samples and aggregated measurement: length, language mix, sender patterns, share of urgent messages. If they diverge, I revise the generator. I also treat synthetic scores as provisional and weight consented and live measures more as they accumulate."
- **Q.** What if reviewers see private content? **A.** "They only see donated or synthetic content under agreements, with access controls, retention limits and redaction of identifiers. For live quality, I use on-device or aggregated signals so that no reviewer reads real users' notifications."

IQ-C25-04 · Intermediate · ai · AI · Microsoft

Prompt. Design the evaluation for an assistant that answers employees' questions from company documents.

Rubric.

- **Baseline:** "Ask employees if the answers are good."
- **Strong:** Builds a question set and evaluates retrieval, correctness and abstention.
- **Standout:** Separates retrieval, groundedness, correctness and abstention; includes permission tests and unanswerable questions; sets release gates with sample-size reasoning; and monitors drift as documents change.

Model answer. The setting is general; nothing here describes any real product. I'd evaluate the pipeline in parts and end to end.

The set. About 600 real questions from search logs and help-desk tickets, stratified by department (HR, IT, finance, legal), with a fifth deliberately unanswerable from the documents, and a few dozen adversarial ones (instructions hidden in a question, requests for documents the asker may not see). Domain owners write the reference answer and the source passage for each.

Measures. **Retrieval:** is the right passage in the top four? **Groundedness:** is every claim supported by the cited passage? **Correctness:** does the answer match the reference? **Abstention:** does it say "not found" on the unanswerable? **Citation validity:** do the cited sources exist and support the claim? **Permission safety:** with test accounts of different access, no content from documents they may not read, tested on every change.

Sizing. To read a confident-wrong rate near 5% within ± 2.5 points needs about 292 cases ($1.96^2 \times 0.05 \times 0.95 \div 0.025^2$), so 600 total supports the overall reading, with a floor of about 100 per department for gross failures and more in the sensitive ones (HR, legal).

Judges. Domain experts score a sample; two scorers check agreement; a model grader screens the rest after calibration with a human sample of what it passes.

Release gate. For example: confident-wrong at most 5% overall and at most 2% in HR, legal and finance; citation validity at least 95%; zero permission leaks in the test; abstention on at least 85% of unanswerable questions; per-department floors. The gate runs on every change to the model, prompt, retrieval or index.

Production. Review 100 sampled answers a week; track "not helpful" reports, escalations and the age of cited documents; watch drift as documents change (an old policy gives a confident wrong answer), and add failures to the "never again" set. Launch department by department, starting where documents are cleanest.

Company note. Microsoft is the setting only: `Inferred` from candidate reports of enterprise-assistant evaluation prompts `[unverified: candidate-reported]`. No Microsoft data or design is claimed.

Follow-ups.

- **Q.** Domain experts are expensive. How do you cut cost? **A.** "Use them for the reference answers and the disputed cases, use a calibrated model grader to extend coverage, and sample rather than review everything. Spend the most expert time where errors cost most (legal, pay, safety)."
- **Q.** How do you detect that documents have become outdated? **A.** "Track document age and last review date, compare cited documents with the current versions, alert when a cited document is replaced, and rerun the eval after major content updates. Surface the age to users and the owner to fix."

IQ-C25-05 · Intermediate · ai · AI · Google

Prompt. Can we use a language model to grade the answers of another language model?

Rubric.

- **Baseline:** "Yes, it saves money," or "no, it can't be trusted."
- **Strong:** Says to calibrate it against human judgments.
- **Standout:** Measures the judge's precision and recall on real failures, explains why accuracy misleads with rare failures, names its biases, and designs a triage with human samples.

Model answer. The setting is general; nothing here describes any real system. Yes, with care: it is a useful tool and a poor final authority until measured.

Calibrate first. Take a sample of outputs judged by trained people, then have the model judge the same outputs against the same rubric, and compare. Do not stop at agreement. Suppose, of 1,000 outputs, humans find 100 bad. The model flags 80 as bad, 70 of them right and 10 wrong. Precision is $70 \div 80 = 87.5\%$; recall is $70 \div 100 = 70\%$; overall accuracy is $(70 + 890) \div 1,000 = 96\%$. The 96% sounds excellent and hides that the judge lets 30% of the bad outputs through.

Known weaknesses. Preference for longer or more fluent answers; trouble verifying facts it was not given; sensitivity to the order of answers shown to it; leniency toward outputs that resemble its own style; mistakes correlated with the system being graded.

Make it better. Give it the source documents and ask claim-by-claim checks; use a separate check per rubric criterion instead of one overall score; include examples of subtle failures in its instructions; use a different model family from the one being graded where possible; randomize answer order. Recalibrate after each change.

Use it as triage. Run it on everything, send the flagged items to humans along with a random sample of the passed ones to estimate the miss rate, and track that miss rate over time. Do not gate a launch on the judge alone for high-stakes use.

Company note. Google is the setting only: Inferred from candidate reports of generated-answer evaluation prompts [unverified: candidate-reported] . No Google data or practice is claimed.

Follow-ups.

- **Q.** What if humans disagree with each other? **A.** "Then the judge has no clear target. Measure human agreement beyond chance first, fix the rubric with scored examples, and use the humans' consensus as the reference. A judge cannot be more consistent than its reference."
- **Q.** Can a model grade itself? **A.** "It is the weakest arrangement: its blind spots are the same in generating and judging. Use a different model, human samples and rule-based checks for anything checkable (does the cited source exist; does the code run)."

IQ-C25-06 · Hard · ai · AI · Meta

Prompt. Suggested replies in a messaging product are accepted 38% of the time. Is the feature working?

Rubric.

- **Baseline:** "Yes, 38% is good adoption."
- **Strong:** Questions whether acceptance measures quality and asks about errors.
- **Standout:** Measures accepted-and-wrong, computes the break-even error rate from time saved and cost of error, reads effects by week, checks sensitive contexts and long-term effects with a holdout, and sets a launch rule.

Model answer. The setting is general; nothing here describes any real product. Acceptance tells me people tapped the suggestion; it does not tell me the suggestion was right, appropriate or helpful. It could reflect a useful feature or one users trust too much.

Check what was accepted. Sample accepted suggestions and have reviewers judge them: wrong facts, wrong tone for the relationship, inappropriate for the moment. Suppose 5% are bad. Track later corrections, follow-up messages and complaints too.

Break-even. Suppose an accepted suggestion saves 18 seconds and a bad one costs the user 6 minutes (360 seconds) in repair or embarrassment. The break-even error rate is $18 \div 360 = 5\%$, since the saving applies to every acceptance and each bad one adds 360 seconds of repair. At 5% bad among accepted the feature is a wash for the user; below it helps, above it hurts. So 38% acceptance with 5% bad is no evidence of value, and I would want the error rate well below 5% with an interval below the break-even.

Time and learning. Read acceptance by week: early novelty inflates it, and users learn which suggestions to trust, so it may fall. Compare with a holdout over several weeks on messaging volume, satisfaction, and relationship-sensitive outcomes (conversations that end abruptly after a suggested reply).

Context risk. Replies in sensitive contexts (condolences, conflict, health, money) cost more when wrong; exclude or restrict suggestions there, and measure by context.

Launch rule. Accepted-and-wrong at most 2% overall and near zero in sensitive contexts; complaints no higher than the holdout; no drop in conversation health measures; and monitoring after launch. What would change my view: evidence that bad suggestions concentrate in a segment or topic, which I can remove while keeping the benefit elsewhere.

Company note. Meta is the setting only: `Inferred` from candidate reports of feature-experiment prompts `[unverified: candidate-reported]`. No Meta data or practice is claimed.

Follow-ups.

- **Q.** How do you estimate the cost of a bad reply? **A.** "Combine measurable proxies (follow-up corrections, time to the next message, reports) with research (interviews and diary studies on how people feel and what they do after a wrong suggestion), and test sensitivity: if the cost doubles or halves, does the launch decision change? If it does, invest in measuring it better."
- **Q.** Users can always edit a suggestion, so why does accepted-and-wrong matter? **A.** "Many accept without editing, especially when it is one tap and looks right. Editing is the safety net only for those who read carefully. The product should make errors easy to see (highlight names, numbers and claims) and not rely on users noticing."

IQ-C25-07 · Hard · ai · AI · Amazon

Prompt. Price an AI assistant add-on for sellers on a marketplace so that it makes money.

Rubric.

- **Baseline:** "Charge a flat monthly fee similar to competitors."
- **Strong:** Computes cost per task and margin and considers heavy users.
- **Standout:** Computes margin by segment, shows how an average hides losses, designs allowance and overage, routing and caps, tests price elasticity, and ties price to seller value.

Model answer. The setting is general; nothing here describes any real product. A flat fee is attractive to sellers and risky because AI cost scales with use.

Cost by segment. Suppose a seller's average use is 150 tasks a month at \$0.04 each (\$6) plus \$1.50 of other cost: \$7.50. A heavy seller (10% of sellers) runs 900 tasks: \$36 plus \$1.50: \$37.50. At a flat \$15 a month: average margin \$7.50 (50%), heavy margin $-\$22.50$. Blended cost: $0.9 \times \$7.50 + 0.1 \times \$37.50 = \$10.50$, so the margin is \$4.50, 30%.

Redesign. Include 400 tasks a month and charge \$0.05 per additional task. The heavy seller then pays $\$15 + 500 \times \$0.05 = \$40$ against a cost of \$37.50: a small positive margin. Blended revenue: $0.9 \times \$15 + 0.1 \times \$40 = \$17.50$, margin $\$17.50 - \$10.50 = \$7.00$, 40%. Average sellers see no change.

Other levers. Route simple tasks to cheaper models (cost per task falls; test quality by route); cache repeated outputs; shorten prompts; offer a higher-priced tier for power users; consider pricing by outcome where value is measurable (a listing that sells).

Value anchor. Price relative to what the seller gains: if the assistant saves a seller five hours a month or lifts conversion, a \$15 fee is small; measure that benefit and use it in the message.

Test. Price experiments by cohort for elasticity (conversion, churn, usage), watch complaints about overage, and monitor margin by cohort monthly. Keep in mind the marketplace context: sellers who feel nickel-and-dimed may leave, so make allowances generous enough that most never see overage.

Company note. Amazon is the setting only: Inferred from candidate reports of pricing and unit-economics prompts [unverified: candidate-reported]. No Amazon data or pricing is claimed.

Follow-ups.

- **Q.** What if model prices fall by half next year? **A.** "Margins improve and competitors may cut prices, so I would not bank the savings in the price; I would keep the allowance structure, reinvest part of the saving in more included usage if the market demands it, and keep tracking cost per task. Plan on current costs and treat declines as upside."
- **Q.** How do you avoid sellers gaming the allowance? **A.** "Metering and rate limits per account, detection of automated resale of the assistant's output, and terms of use. Watch for accounts whose usage pattern looks like resale, and review heavy users to see whether they are valuable sellers or abusers."

IQ-C25-08 · Hard · ai · AI · Stripe

Prompt. An AI agent handles payment disputes and refunds for support. How do you decide how much autonomy to give it, and when to give it more?

Rubric.

- **Baseline:** "Let it handle small refunds on its own."
- **Strong:** Sets approval levels by amount and error cost.
- **Standout:** Defines promotion evidence with sample sizes, per-band limits, demotion triggers, audit and reversibility, and weighs the economics against the cost of error.

Model answer. The setting is general; nothing here describes any real system. Autonomy should be earned per action and per amount band, with evidence and a way back (FW-C25-05).

Start low. At first the agent drafts the decision and the reply; a person approves each one. This gathers data on how often the agent is right, for each type and amount.

Promotion evidence. To promote small refunds (under \$20) to autonomous, require: at least 300 supervised cases in that band with no errors (the upper 95% bound on the error rate is then about 1%, by the rule of three), or about 600 for a 0.5% bound; no severe incident; low override rate by reviewers; reversibility (a refund can be reversed or limited); and monitoring and a kill switch in place. Each amount band and each case type (duplicate charge, item not received, fraud claim) earns its own evidence; reliability on \$10 duplicate charges says little about \$500 fraud disputes.

Limits that bound the risk. Caps on amount per refund and per customer per month, a daily total, a rule that suspicious patterns go to a person, an audit log of every action with the evidence used, and a short reversal window.

Demotion. A written rule: if the error rate in a week exceeds 1% on at least 100 cases, or any severe incident occurs (a refund to the wrong party, a fraud loss above a set amount), drop that band back to approval and investigate.

Economics. Suppose 50,000 refunds a month, and automating a refund saves \$2 of handling: \$100,000 a month. If 0.5% are wrong at an average cost of \$30, errors cost $250 \times \$30 = \$7,500$ a month, a small fraction of the savings, so promotion pays when the evidence supports a 0.5% bound. For larger amounts the cost of an error rises with the amount, and the evidence bar rises with it.

Fraud and abuse. Refund agents attract manipulation (a customer's message instructing the agent); treat message text as untrusted, never let it change limits, and require extra checks for risky signals.

Company note. Stripe is the setting only: Inferred from candidate reports of automation-with-money prompts [unverified: candidate-reported]. No Stripe data or practice is claimed.

Follow-ups.

- **Q.** What if the agent is very accurate but a rare error is severe? **A.** "Separate error types by severity: a high accuracy on routine errors does not allow autonomy where one error is catastrophic. Cap the amount so the worst case is bounded, require approval for the severe categories regardless of accuracy, and track severe errors with a zero-tolerance trigger."
- **Q.** How do you know reviewers are not rubber-stamping approvals? **A.** "Measure the override rate and the time per approval, seed the queue with known-bad cases to see whether reviewers catch them, and rotate reviewers. If almost nothing is overridden and seeded errors are missed, approvals are not adding safety, and the process needs redesign."

IQ-C25-09 · Hard · ai · AI

Prompt. Your company sells an AI product in several regions. How do you approach regulation and compliance?

Rubric.

- **Baseline:** "Ask legal to check it."
- **Strong:** Classifies by risk and notes regional differences.
- **Standout:** Triage each use by risk tier and market, uses current dates carefully (including deferrals), builds documentation and transparency into the product, and sets owners and launch implications, without giving legal advice.

Model answer. I am not a lawyer and rules change quickly, so I'd involve legal and policy from the start and check current texts [as of 2026-10]. My job as PM is to give them a clear picture and build the product to meet what they decide.

Triage (FW-C25-07). List each AI use and each market. Classify by risk tier: prohibited uses; high-risk uses (employment decisions, credit, education, essential services, safety components); limited-risk uses with transparency duties (chatbots must disclose that the user is dealing with a machine; generated content may need labeling); minimal risk (a spam filter). Take an example: a customer-service chatbot is typically limited-risk, so the main duty is disclosure; a resume-screening feature is typically high-risk, with obligations on documentation, human oversight, data quality, and testing for bias.

Know the dates, and read deferrals carefully. In the European Union, for example, prohibited practices have applied since February 2025, obligations for general-purpose model providers since August 2025, and transparency duties for chatbots and generated content from 2 August 2026 (with a transition to 2 December 2026 for machine-readable marking by generative systems already on the market); the obligations for high-risk systems, originally due 2 August 2026, were deferred by Regulation (EU) 2026/1744 to 2 December 2027 for stand-alone systems and 2 August 2028 for AI embedded in regulated products [V: Regulation (EU) 2026/1744 on EUR-Lex; legal summaries for the dates]. A deferral is not an exemption: the work of documentation and testing takes time, and transparency duties may already apply.

Build in what is required. Disclosure that the user is interacting with an AI; labeling of generated content; human review for significant decisions; records of the system's purpose, data, evaluation results, known limits and monitoring (a model card or equivalent); logs for audit; data protection (what is collected, retention, user rights).

Other regions. In the United States, existing consumer-protection and anti-discrimination laws apply and some states and cities have AI-specific rules [V: US: April 2023 joint statement by EEOC, CFPB, FTC and DOJ on existing law; NYC Local Law 144; other state rules not enumerated]; in India, data protection law, non-binding AI governance guidelines and binding IT Rules amendments on synthetic content apply [V: India: non-binding MeitY guidelines and binding IT Rules amendments, per law-firm summaries, 2026]; in China,

specific rules cover recommendation algorithms and generative services [V: China: algorithm, deep synthesis and generative AI rules, per law-firm summaries]. A feature may need different designs, or a delayed launch, in different markets.

Operate it. Assign owners (legal, policy, security, product), put compliance gates into the launch checklist, and review when rules change.

Follow-ups.

- **Q.** A feature is high-risk in one market only. What do you do? **A.** "Decide with legal whether to launch there with the required controls, to launch a limited version (for example with a person making the final decision), or to delay that market; do not weaken the design everywhere by default, and do not ship a single global behavior that is non-compliant somewhere. Keep one codebase with market-specific configuration where possible."
- **Q.** How do you keep up with changing rules? **A.** "Name an owner (policy or legal) who tracks changes and a quarterly review with product; keep a register of AI uses with their tiers and obligations; and treat compliance requirements as inputs to the roadmap with lead time, not as last-minute gates."

IQ-C25-10 · Intermediate · ai · AI

Prompt. What is the moat of an AI writing assistant if the underlying models keep improving?

Rubric.

- **Baseline:** "Our AI is better than the competition's."
- **Strong:** Names data, distribution or workflow as sources of advantage.
- **Standout:** Tests each source for durability, separates what compounds from what does not, notes that model access is not a moat, and says what to build toward.

Model answer. I'd be honest: the model itself is unlikely to be the moat if others can access the same or better models. So I'd test each candidate source with the sources-of-advantage test (FW-C18-05).

Weak sources. A clever prompt (easy to copy and to replace); a thin wrapper around a general model; the choice of model (rivals can switch); novelty.

Possible sources. (1) *Distribution:* if the assistant sits inside the tools where people already write (email, documents), it reaches users that a standalone rival must win. (2) *Workflow integration and switching costs:* the more it holds a person's or team's templates, style guides, past documents and approvals, the harder it is to leave. (3) *Proprietary data and feedback loops:* data that others lack and that improves with use (what edits users make to suggestions, house style) can compound, though a stronger general model may erase the advantage if the task does not need your data. (4) *Evaluation capability:* a mature test set and monitoring let me improve faster and ship safely; a new entrant starts from nothing. (5)

Trust and accountability: in domains with costly errors (legal, medical, finance), a record of reliability, audit trails and clear responsibility matters. (6) *Cost position:* routing and caching deliver the same quality for less, which allows a better price or margin.

What compounds. Workflow depth, data from use and trust compound; prompts and model choice do not. So I'd invest in becoming part of how teams write: shared style guides, review flows, integrations, and an eval that makes each model upgrade safe to adopt.

What I'd watch. Whether competitors with the same models match our quality; whether the platforms we depend on add the feature natively; and whether our retention comes from habit and integration or from novelty.

Follow-ups.

- **Q.** What if the platform you sit inside builds the same thing? **A.** "Then the product must be better where the platform cannot or will not go: a specialized workflow, a vertical's compliance and data, deeper integration across several tools the platform does not span, or a different audience. Otherwise the honest answer is that the feature is a feature, not a company, and I would plan accordingly."
- **Q.** Does proprietary data always give a moat? **A.** "No. If a general model can do the task well without your data, the advantage vanishes; if the data is hard to get, improves the product measurably as it grows, and is legal and consented to use, it can last. Test it: does a model with our data beat a strong general model by a margin users notice?"

12. Exercises

Ten exercises. Each has a model solution and an alternative. Figures are assumptions unless marked, and the arithmetic was computed.

EX-C25-01 · Size an eval set

Problem. You expect a 5% error rate and want it within ± 2 points at 95% confidence. How many cases do you need? If you want to read five languages separately at that precision, how many in total? What can you say from a set of 30 cases with no errors?

Context. Planning an evaluation.

Data. $n = 1.96^2 \times p \times (1 - p) \div \delta^2$.

Constraints. Five minutes.

Expected approach. FW-C25-02.

Solution. $n = 3.84 \times 0.05 \times 0.95 \div 0.0004 =$ about 456. Five languages at that precision: about 2,280. With 30 cases and no errors, the 95% upper bound on the error rate is about $3 \div 30 = 10\%$: the set cannot show that the rate is low. To claim below 1% you need about 300 clean cases.

Alternative solution. Accept a wider margin (± 3 points) for low-traffic languages: n falls to about 203 each.

Tradeoffs. Tighter margins and more slices cost more reviewers' time; spend where errors cost most.

EX-C25-02 · Agreement beyond chance

Problem. Two reviewers label 200 outputs as bad or good. They agree both bad on 10, both good on 170; A says bad and B good on 10; A says good and B bad on 10. Compute raw agreement, chance agreement and kappa, and say what to do.

Context. Checking a rubric.

Data. $\text{Kappa} = (\text{observed} - \text{expected}) \div (1 - \text{expected})$.

Constraints. Six minutes.

Expected approach. C25.1 and Worked example 1.

Solution. Observed agreement: $(10 + 170) \div 200 = 90\%$. Each reviewer flags 20 of 200 (10%), so expected agreement is $0.1 \times 0.1 + 0.9 \times 0.9 = 82\%$. $\text{Kappa} = (0.90 - 0.82) \div 0.18 = 0.44$: moderate. The rubric is ambiguous: review the 20 disagreements, add scored examples that settle them, retrain reviewers, and remeasure on a new sample. Do not rely on a 90% raw agreement.

Alternative solution. Have a third reviewer break ties, and record the rationale to refine the rubric.

Tradeoffs. Rubric work costs time and reduces noise in every later measurement.

EX-C25-03 · Calibrate a model grader

Problem. Of 250 outputs, humans find 50 bad. A model grader flags 40, of which 30 are among the 50. Compute precision, recall and accuracy, and propose a triage design.

Context. Using a model judge.

Data. As above.

Constraints. Six minutes.

Expected approach. Worked example 2.

Solution. True positives 30, false positives 10, false negatives 20, true negatives 190. Precision $30 \div 40 = 75\%$. Recall $30 \div 50 = 60\%$. Accuracy $(30 + 190) \div 250 = 88\%$. The 88% hides that 40% of the bad outputs pass. Triage: humans review the 40 flagged plus a random sample (say 15%) of the 210 passed, to estimate the miss rate (20 of 210 is 9.5%); improve the grader with claim-by-claim checks and examples of subtle failures; recalibrate.

Alternative solution. Use two different graders and send disagreements to humans.

Tradeoffs. More human review raises cost and certainty; choose by the cost of a missed failure.

EX-C25-04 · Acceptance and break-even

Problem. A feature shows 500,000 suggestions; 40% are accepted. Each acceptance saves 30 seconds. 6% of accepted suggestions are wrong, and each wrong one costs 8 minutes. Compute the net time and the break-even error rate. What do you recommend?

Context. An online test readout.

Data. As above.

Constraints. Six minutes.

Expected approach. Worked example 3.

Solution. Accepted: 200,000. Time saved: $200,000 \times 0.5 \text{ minute} = 100,000 \text{ minutes}$. Wrong accepted: $6\% \times 200,000 = 12,000$; cost $12,000 \times 8 = 96,000 \text{ minutes}$. Net: +4,000 minutes, barely positive. Break-even error rate: $0.5 \div 8 = 6.25\%$, so 6% is almost at break-even and uncertainty about the 6% (a sample of a few hundred has a wide interval) means the feature may be harmful. Recommend: improve the error rate (restrict contexts, highlight facts), re-measure with a larger sample, and require the interval's upper end to be below the break-even before launch.

Alternative solution. Show suggestions only when the model is confident and measure the error rate in that subset.

Tradeoffs. Restricting suggestions lowers acceptance and raises net value.

EX-C25-05 · Layer arithmetic

Problem. There are 200,000 attack attempts a day. An input filter stops 70%, the model's refusal stops 85% of what passes, and an output filter stops 60% of what remains. Compute successful attacks. Then compare improving the output filter to 80% against improving the input filter to 85%.

Context. Safety planning.

Data. As above.

Constraints. Six minutes.

Expected approach. Worked example 4.

Solution. Passing the input filter: 30%; passing refusal: 15% of those; passing the output filter: 40% of those. Success: $200,000 \times 0.3 \times 0.15 \times 0.4 = 3,600$ a day. Output filter at 80%: $200,000 \times 0.3 \times 0.15 \times 0.2 = 1,800$. Input filter at 85%: $200,000 \times 0.15 \times 0.15 \times 0.4 = 1,800$. Both halve the leakage; the better choice depends on cost and on the effect on legitimate users (over-refusal). Check each filter's false-positive rate on benign requests before choosing.

Alternative solution. Add a rate limit that halves the attempts from repeat attackers, which halves all of the above.

Tradeoffs. Stricter filters block more legitimate use; measure both.

EX-C25-06 · Unit economics

Problem. A \$15 per-user monthly AI add-on: the average user runs 200 tasks at \$0.02 plus \$1.50 other cost; heavy users (15% of users) run 1,000 tasks. Compute the margin for each segment and blended, and propose a change.

Context. Pricing review.

Data. As above.

Constraints. Seven minutes.

Expected approach. FW-C25-06.

Solution. Average user: cost $200 \times \$0.02 + \$1.50 = \$5.50$; margin \$9.50 (63%). Heavy user: $1,000 \times \$0.02 + \$1.50 = \$21.50$; margin $-\$6.50$. Blended cost: $0.85 \times \$5.50 + 0.15 \times \$21.50 = \$4.675 + \$3.225 = \$7.90$; blended margin \$7.10 (47%). The heavy segment loses money and takes a quarter of the average user's margin. Options: include 500 tasks with overage at \$0.03 (the heavy user pays $\$15 + 500 \times \$0.03 = \$30$, margin \$8.50); route simple tasks to a cheaper model; a higher tier. Test elasticity before changing.

Alternative solution. Raise the price to \$18 for all; simpler and affects everyone.

Tradeoffs. Overage protects margin and creates friction; make allowances generous for typical users.

EX-C25-07 · Read a regression by slice

Problem. A suite of 500 cases (400 English, 100 Spanish) scores before a model update: English 384 pass (96%), Spanish 96 pass (96%). After: English 384 pass, Spanish 84 pass. Compute the overall pass rates and say what to do.

Context. A launch gate.

Data. As above.

Constraints. Five minutes.

Expected approach. CS-C25-01.

Solution. Before: $(384 + 96) \div 500 = 96\%$. After: $(384 + 84) \div 500 = 93.6\%$. The overall figure fell 2.4 points, which could pass a gate set at 93%; but all of the loss is in Spanish, which fell from 96% to 84% (12 points). A per-slice gate (for example, no language below 92%) would block the release. Investigate why Spanish regressed (tokenization, prompt wording, retrieval of Spanish documents), fix, and rerun.

Alternative solution. Ship the update for English only while Spanish is fixed, if the system can route by language.

Tradeoffs. Per-slice gates block more releases and prevent quiet harm to smaller groups.

EX-C25-08 · A trust progression plan

Problem. Write promotion and demotion rules for letting an agent send automatic order-status updates and answer simple returns questions, and say how many supervised cases justify promotion if you want a 95% upper bound of 1% on the error rate.

Context. Autonomy design.

Data. Upper bound about $3 \div n$ for zero errors in n cases.

Constraints. Six minutes.

Expected approach. FW-C25-05.

Solution. For a 1% bound with no errors, $n = 3 \div 0.01 = 300$ supervised cases per category (about 600 for a 0.5% bound). Promotion rule: promote a category (order status; return-eligibility answers) when it has at least 300 supervised cases with no errors, no severe incident, a reviewer override rate under 2%, a reversal path, and monitoring and a kill switch in place. Demotion: if the weekly error rate exceeds 1% on at least 100 cases, or one severe incident occurs, return the category to approval and investigate. Limits: only the customer's own orders, no changes to payment or address, a daily action cap, and a log of every action.

Alternative solution. Use a sequential test that allows earlier promotion if there are no errors and demotion if errors appear.

Tradeoffs. A higher evidence bar delays benefits; a lower one raises risk.

EX-C25-09 · Compliance triage

Problem. Classify these uses by likely risk tier under a risk-based regime such as the EU AI Act and say the main obligation: (a) a customer-service chatbot; (b) a resume-screening tool; (c) a spam filter; (d) a system that scores people's creditworthiness; (e) a tool that infers employees' emotions from video at work.

Context. Product planning across markets.

Data. None.

Constraints. Six minutes.

Expected approach. FW-C25-07.

Solution. [V: EU AI Act as amended by Regulation (EU) 2026/1744] (a) Limited risk: disclose that users are talking to an AI. (b) High-risk (employment): documentation, data quality, human oversight, testing for bias, and logs; applicable dates were deferred in the EU to December 2027, but preparation takes time. (c) Minimal risk: no special obligations. (d) High-risk (access to essential services such as credit): the same family of obligations as (b). (e) Likely prohibited in workplaces under the EU's rules on emotion recognition, with narrow medical or safety exceptions (the ban has applied since 2 February 2025 and covers recruitment) [V: EU AI Act Article 5(1)(f)]: do not build or deploy it there. Every classification must be confirmed with counsel and the current text.

Alternative solution. For (b) and (d), design for human decision-making with the model as one input, which also supports oversight duties.

Tradeoffs. Higher tiers add cost and delay; classification mistakes cost more.

EX-C25-10 · A timed full answer

Problem. Give a 30-minute answer to: "Design the evaluation and launch plan for an AI agent that handles customer-support conversations for a bank."

Context. Self-scored; record yourself. State assumptions aloud.

Data. None. Label what you assume.

Constraints. Use the 30-minute budget in section 4.

Expected approach. FW-C25-08, scored against the six behaviors in section 1.

Solution. An outline of a strong answer. Cost of error: wrong information about fees, deadlines or fraud steps creates liability and harm, and the company answers for what the agent says (C24, CS-C24-02). Eval: 1,000 real conversations (anonymized, with consent and legal review), sliced by topic (cards, loans, fraud, complaints), language and customer type, plus adversarial and unanswerable cases and scripted fraud attempts; rubric (supported by policy text, correct escalation, tone, no disclosure of other customers' data); two reviewers with a kappa target of at least 0.8; a calibrated model grader for triage; pass bar: at most 1% unsupported claims overall, none in fees and fraud steps, zero leaks, and 100% correct escalation of fraud and complaint cases in the test; per-language gates. Metrics: resolution without escalation, accepted-and-wrong (sampled), customer satisfaction, complaints, cost per conversation, time to resolution; guardrails on regulator-reportable events. Safety: layers (scope: no advice, no money movement; input filtering for injection; grounded answers quoting current policy; output checks for account data; limited permissions; human review queue; monitoring), red-team by harm category including social-engineering attempts, kill switch and incident owner.

Autonomy: starts as draft for agents; promotes informational answers after 300 supervised cases per category with no errors; money movement stays with people. Economics: cost per conversation against the cost of a human-handled one; routing; heavy-case limits.

Regulation: the AI-interaction disclosure, record keeping, bank regulators' expectations on model risk and complaints handling, data protection [V: AI Act Article 50 as amended by Regulation (EU) 2026/1744; the other items were not rechecked]. Rollout: shadow, then staff-facing, then a small customer group, with a long-term holdout on satisfaction and complaints.

Alternative solution. Start with the assistant supporting human agents (draft replies and policy lookup) and only later face customers directly.

Tradeoffs. Agent assistance carries lower risk and lower savings than customer-facing automation.

13. Practice protocol

Timed drills (three a week for three weeks).

- *Week 1: evaluation basics.* Use IQ-C25-01, IQ-C25-02, EX-C25-01, EX-C25-02 and EX-C25-03. Compute sample sizes and kappa by hand and say the eval design card aloud for a new feature.
- *Week 2: metrics, graders and design.* Use IQ-C25-04, IQ-C25-05, IQ-C25-06, EX-C25-04, EX-C25-05 and EX-C25-07. Compute a break-even and a leakage figure, and read a regression by slice.
- *Week 3: autonomy, economics, regulation and full answers.* Use IQ-C25-03, IQ-C25-07, IQ-C25-08, IQ-C25-09, IQ-C25-10, EX-C25-06, EX-C25-08, EX-C25-09 and EX-C25-10. Run a full 30-minute answer with a partner who raises a failure or a regulation midway.

With a partner. Ask your partner to say "Acceptance is 45%," "The judge agrees 90% of the time" and "We tested it on 50 examples" at random. Practice answering with a break-even, recall on failures and a sample-size calculation.

Build a mini eval. For a task you know, write 30 cases and a three-line rubric, ask a colleague to score ten independently, and compute agreement beyond chance. Revise the rubric once and measure again.

Feedback checklist (yes, partly, no).

1. Did I design the eval before the feature, with real, sliced and adversarial cases?
2. Did I size the set and report intervals?
3. Did I check reviewer agreement beyond chance and calibrate any model grader?
4. Did I separate output quality from product behavior and measure accepted-and-wrong?
5. Did I plan shadow mode, a staged launch and a holdout?
6. Did I layer safety, red-team by category and say what a clean test proves?
7. Did I set autonomy by evidence with limits and demotion rules?
8. Did I compute cost per task and margin by segment?
9. Did I classify risk and regulation by market, with current dates?
10. Did I close with a decision and what would change it?

Answer journal. For each drill record the date, the prompt, the pass bar and sample size you stated, the metric you distrusted, the safeguards, your score out of ten and the point you rushed.

14. Implementation playbook

Apply the chapter on real work over the next three weeks. If you do not work on AI features, apply it to an AI tool your team uses. Each deliverable doubles as an interview story.

Days 1 to 3. Pick an AI feature or tool and write its eval design card (FW-C25-01): failure types, cases, rubric, judges and a pass bar tied to the cost of error. Review it with the engineer or a colleague and note what is missing.

Week 1. Collect 50 real cases (including awkward ones). Score them yourself and with a colleague; compute raw agreement and kappa. Fix the rubric and remeasure.

Week 1. Compute the sample size you would need for the rate and slices you care about, and compare it with what exists today.

Week 2. List the feature's metrics by layer (FW-C25-03) and identify the one that could mislead (for example, acceptance); design a way to measure accepted-and-wrong and compute a break-even.

Week 2. Write the safety layers (FW-C25-04) for the feature, estimate each layer's share, and compute the leakage; name the kill switch owner.

Week 3. Build the unit-economics sheet by user segment (FW-C25-06), and the risk-tier note for the markets you serve (FW-C25-07), reviewed with finance and legal.

Deliverables that become stories: the eval design card with the kappa check (a time you made quality measurable), the break-even calculation (a time you distrusted a metric and quantified why), and the layered safety and economics notes (a time you designed for failure and for margin). Keep the numbers real, and keep what you did separate from what the team did.

15. Key takeaways and further reading

Key takeaways

1. Build the evaluation before the feature: real, sliced and adversarial cases, a rubric reviewers agree on beyond chance, a pass bar tied to the cost of error, run on every change and read by slice, with a hidden test set and a "never again" set from production.
2. Size and read evaluations honestly: cases needed grow with precision and slices ($n = 1.96^2 \times p \times (1 - p) \div \delta^2$), small clean sets cannot certify low error rates (about $3 \div n$), and a model grader must be calibrated on recall for real failures, not agreement.
3. Separate model quality from product value and distrust acceptance: measure accepted-and-wrong and compute the break-even error rate from time saved and cost of error; read effects by week; keep a long-term holdout and use shadow mode.
4. Design safety in layers and test it adversarially: multiply what each layer lets through, state what a clean red-team result proves, plan monitoring, a kill switch and incident response, and earn autonomy with evidence, limits and demotion rules.
5. Make the product viable and lawful: compute cost per task by user segment (heavy users can turn margin into loss), classify uses by risk tier and market with current dates, keep documentation, and build defensibility from what compounds (workflow, data, trust, evaluation), not from a prompt or model choice.

Further reading

- Chip Huyen, *AI Engineering* (2025), for evaluation, monitoring and the economics of building on foundation models [V: O'Reilly, first release 4 December 2024, in print January 2025, ISBN 9781098166304] .
- Ron Kohavi, Diane Tang and Ya Xu, *Trustworthy Online Controlled Experiments* (2020), for experiments with guardrails, novelty and long-term effects [V: Cambridge University Press, 2 April 2020, ISBN 9781108724265; Twyman's law is chapter 3] .
- Peter Lee, "Learning from Tay's introduction," Official Microsoft Blog (25 March 2016; blogs.microsoft.com/blog/2016/03/25/learning-tays-introduction/). The primary source for CS-C25-02.
- Microsoft's Responsible AI Standard and guidelines for human-AI interaction, and similar published frameworks from other companies, as examples of governance practice [V: Microsoft Responsible AI Standard] .
- The text of the EU AI Act and the 2026 amending package, and official guidance from the European Commission, read for current dates (not legal advice) [V: Regulation (EU) 2026/1744 on EUR-Lex] .
- The NIST AI Risk Management Framework (AI RMF 1.0, NIST AI 100-1, January 2023, with a Generative AI Profile, NIST AI 600-1, July 2024), as a structured approach to identifying and managing AI risks [V: NIST AI 100-1, 26 January 2023; Generative AI Profile NIST AI 600-1, 26 July 2024; NIST's page says AI RMF 1.0 is being revised, checked October 2026] .
- Margaret Mitchell and colleagues, "Model Cards for Model Reporting," *Proceedings of FAT '19** (2019), pp. 220 to 229, on documenting a model's intended use and evaluation [V: FAT* '19, 29 to 31 January 2019, pp. 220 to 229, doi 10.1145/3287560.3287596] .